

单位代码： 10293 密 级： _____

南京邮电大学

硕士学位论文



论文题目： 基于深度学习的源代码漏洞检测方法研究与实现

学 号	1020041122
姓 名	王璇
导 师	陈燕俐
学 科 专 业	计算机科学与技术
研 究 方 向	计算机应用
申请学位类别	工学硕士
论文提交日期	2023 年 6 月

Research and Implementation of Source Code Vulnerability Detection Method Based on Deep Learning

Thesis Submitted to Nanjing University of Posts and
Telecommunications for the Degree of
Master of Science in Engineering



By

Wang Xuan

Supervisor: Prof. Chen Yanli

June 2023

摘要

随着计算机软件的发展和应用日益广泛, 计算机系统使用量呈指数式增长, 软件漏洞问题威胁着计算机系统的安全性。一方面, 由于现在软件的多样性和复杂性, 用户在使用时会产生不同的情形, 从而产生不同的漏洞问题, 导致漏洞数量增加。另一方面, 计算机软件的开发人员在编写代码时, 会无意留下漏洞被攻击者利用。漏洞检测方法作为一种计算机算法, 可以通过一些学习模型, 学习漏洞函数的相关特征, 检测出软件源代码是否有漏洞。但是, 单一的学习模型在实际应用过程中对使用场景的要求比较高, 且对漏洞特征的表征能力和检测效率有限, 有着很高的漏报率和误报率。针对这一问题, 可使用组合模型和多特征融合方法来解决。因此, 本文基于深度学习的源代码漏洞检测方法研究具有重要意义。

围绕基于深度学习的源代码漏洞检测方法, 本文的主要工作如下:

(1) 为实现漏洞函数的源代码文本深度表征, 本文提出一种基于 DistilBert-LSTM 和多项朴素贝叶斯(DistilBert-LSTM Multinomial Naive Bayes, DBM-MNB)的漏洞检测方法, 将 DBM 和 MNB 两部分进行组合实现漏洞检测。该方法对源代码生成相应的序列, 通过 DistilBert-LSTM 挖掘漏洞的局部关键特征和全局时间特征, 深度挖掘漏洞语句间的依赖关系, 并得出漏洞的存在性概率; 针对漏洞检测过程中的难样本, 该模型通过多项朴素贝叶斯进行优化检测, 使用 TF-IDF 矢量化器进行数据预处理, 并执行卡方检验进行特征选择, 将所得输出至多项朴素贝叶斯分类器中进行检测, 以获得最终的漏洞检测结果。对比实验结果表明, DBM-MNB 模型可提升漏洞特征挖掘的深度, 且相较于当前部分组合学习模型呈现更优的性能指标。

(2) 为实现漏洞函数的全面表征, 本文提出一种基于多特征融合(CHR-CodeBERT)的源代码漏洞检测方法。该方法在分层面和垂直面提取源代码的多种特征, 包括卷积神经网络特征、分层注意力特征和循环神经网络特征。从单词、句子和文档三个方面的细粒度对漏洞进行特征工程, 构建成三个特征矩阵, 通过加性注意力机制进行融合得到一个特征矩阵。同时, 对源代码序列添加特殊标记、Padding 处理以及生成掩码等一系列操作, 构建 CodeBERT 的特征矩阵。将以上两个特征矩阵输入到 CodeBERT 模型进行分类, 达到漏洞检测的效果。对比实验表明, CHR-CodeBERT 相较于当前部分单特征漏洞检测模型呈现更优的综合性能指标。

(3) 基于上述两种漏洞检测方法, 本文设计了基于深度学习的源代码漏洞检测系统。该系统主要包括文件上传模块、源代码处理模块、漏洞检测模块和结果显示模块。文件上传模块可以上传单文件、多文件和压缩文件的源代码数据; 源代码处理模块对上传成功的文件进行清洗和预处理操作, 将源代码表征成模型可以识别的形式; 漏洞检测模块调用 DBM-MNB

模型和 CHR-CodeBERT 模型对源代码进行检测，检测该源代码是否有漏洞，并将检测结果转储；结果显示模块将漏洞检测结果进行可视化，并通过查询模块实现历史数据查询和下载保存。通过系统测试结果表明，该系统具有较强的实用性。

关键词：深度学习，源代码表征，漏洞检测，语言模型

Abstract

With the development and application of computer software, the use of computer systems is increasing exponentially, and the problem of software vulnerabilities threatens the security of computer systems. On the one hand, due to the diversity and complexity of current software, users will have different situations when using it, which will cause different vulnerability problems, resulting in an increase in the number of vulnerabilities. On the other hand, when computer software developers write codes, they will unintentionally leave loopholes to be exploited by attackers. As a computer algorithm, the vulnerability detection method can learn the relevant characteristics of the vulnerability function through some learning models, and detect whether there is a vulnerability in the software source code. However, a single learning model has relatively high requirements for usage scenarios in the actual application process, and has limited representation ability and detection efficiency for vulnerability features, and has a high rate of false negatives and false positives. To solve this problem, a combination of model and multi-feature fusion method can be used to solve it. Therefore, the research on source code vulnerability detection based on deep learning in this paper is of great significance.

Around the source code vulnerability detection method based on deep learning, the main work of this paper is as follows:

(1) In order to realize the deep representation of the source code text of the vulnerability function, this paper proposes a vulnerability detection method based on DistilBert-LSTM and Multinomial Naive Bayes (DistilBert-LSTM Multinomial Naive Bayes, DBM-MNB), which combines the two parts of DBM and MNB to realize vulnerability detection. This method generates the corresponding sequence for the source code, uses DistilBert-LSTM to mine the local key features and global time characteristics of the vulnerability, deeply mines the dependency relationship between the statement of the vulnerability, and obtains the existence probability of the vulnerability; for difficult samples in the vulnerability detection process, the model optimizes detection through Multinomial Naive Bayes, uses TF-IDF vectorizer for data preprocessing, and performs chi-square test for feature selection, and outputs the obtained results to multiple naive Bayesian classifiers to obtain the final vulnerability detection results. The comparative experimental results show that the DBM-MNB model can improve the depth of vulnerability feature mining, and presents better performance indicators than the current part of the combination learning model.

(2) In order to achieve a comprehensive characterization of vulnerable functions, this paper proposes a source code vulnerability detection method based on multi-feature fusion (CHR-CodeBERT). This method extracts multiple features of source code in both hierarchical and vertical planes, including convolutional neural network features, hierarchical attention features, and recurrent neural network features. Feature engineering is performed on vulnerabilities from three aspects of words, sentences, and documents, and three feature matrices are constructed, which are fused through an additive attention mechanism to obtain a feature matrix. At the same time, a series of operations such as adding special marks, padding processing, and generating masks to the source code sequence construct the feature matrix of CodeBERT. Input the above two feature matrices into the CodeBERT model for classification to achieve the effect of vulnerability detection. The comparative implementation shows that CHR-CodeBERT presents better comprehensive performance indicators than some current single-feature vulnerability detection models.

(3) Based on the above two vulnerability detection methods, this paper designs a source code vulnerability detection system based on deep learning. The system mainly includes file upload module, source code processing module, vulnerability detection module and result display module. The file upload module can upload the source code data of a single file, multiple files and compressed files; the source code processing module cleans and preprocesses the successfully uploaded files, and represents the source code into a form that can be recognized by the model; the vulnerability detection module calls DBM-MNB model and CHR-CodeBERT model detect the source code, detect whether the source code has loopholes, and dump the detection results; the result display module visualizes the vulnerability detection results, and realizes historical data query and download through the query module save. The system test results show that the system has strong practicability.

Key words: Deep learning, Source code representation, Vulnerability mining, Language model

目 录

第一章 绪论.....	1
1.1 研究背景及意义.....	1
1.2 课题来源及研究内容.....	2
1.3 章节安排.....	2
第二章 相关技术研究	4
2.1 基础知识.....	4
2.1.1 源代码漏洞原理介绍.....	4
2.1.2 静态漏洞检测技术.....	5
2.1.3 动态漏洞检测技术.....	6
2.2 基于神经网络的漏洞检测方法	7
2.3 基于 Transformer 的漏洞检测方法.....	10
2.4 基于深度学习表征的漏洞检测方法	10
2.5 本章小结.....	12
第三章 基于深度学习的源代码漏洞检测系统设计	14
3.1 系统总体设计.....	14
3.1.1 系统需求分析.....	14
3.1.2 系统总体架构.....	14
3.2 系统前后端设计.....	15
3.2.1 前后端框架与请求/响应设计	15
3.2.2 系统前端流程.....	17
3.2.3 系统后端流程.....	18
3.3 系统功能模块设计.....	19
3.3.1 文件上传模块.....	20
3.3.2 源代码预处理模块.....	21
3.3.3 漏洞检测模块.....	21
3.3.4 结果显示模块.....	21
3.4 数据库设计.....	22
3.5 本章小结.....	22
第四章 基于 DistilBert-LSTM 与多项朴素贝叶斯的源代码漏洞检测方法	24
4.1 问题分析.....	24
4.2 DBM-MNB 模型框架	25
4.2.1 数据预处理.....	25
4.2.2 DBM 模型.....	26

4.2.3 MNB 模型.....	28
4.3 实验与分析.....	30
4.3.1 实验数据集.....	30
4.3.2 实验设置.....	30
4.3.3 评价指标.....	31
4.3.4 对比实验与结果分析.....	32
4.4 本章小结.....	34
第五章 基于多特征融合的源代码漏洞检测方法	35
5.1 问题分析.....	35
5.2 模型框架.....	36
5.2.1 模型总体框架.....	36
5.2.2 数据预处理.....	37
5.2.3 CNN 特征提取模块	38
5.2.4 HAN 特征提取模块	39
5.2.5 RNN 特征提取模块	40
5.2.6 基于加性注意力机制的特征融合模块	40
5.2.7 CodeBERT 分类模块	40
5.3 实验与分析.....	42
5.3.1 实验数据集.....	42
5.3.2 实验设置.....	43
5.3.3 评价指标.....	43
5.3.4 对比实验与结果分析.....	43
5.4 本章小结.....	47
第六章 系统实现与测试	48
6.1 系统环境.....	48
6.1.1 硬件环境.....	48
6.1.2 软件环境.....	48
6.2 系统方法实现.....	49
6.2.1 后端检测模块.....	49
6.2.2 前端结果可视化模块.....	50
6.3 系统功能测试.....	51
6.3.1 文件上传.....	51
6.3.2 源代码数据预处理.....	53
6.3.3 源代码漏洞检测.....	53
6.3.4 漏洞检测结果存储及可视化.....	54
6.4 本章小结.....	56
第七章 总结与展望	57
7.1 总结.....	57

7.2 展望.....	57
参考文献.....	59

专用术语注释表

缩略词说明:

TF-IDF	Term Frequency–Inverse Document Frequency	词频-逆向文件频率
CNN	Convolutional Neural Network	卷积神经网络
HAN	Hierarchical Attention Networks	分层注意力网络
RNN	Recurrent Neural Network	循环神经网络
LSTM	Long Short-Term Memory	长短期记忆网络
BiLSTM	Bi-directional Long Short-Term Memory	双向长短期记忆网络
SQL	Structured Query Language	结构化查询语言
HTML	HyperText Mark-up Language	超文本标记语言
CVE	Common Vulnerabilities & Exposures	通用漏洞披露
AST	Abstract Syntax Tree	抽象语法树
CFG	Control Flow Graph	控制流图
NLP	Natural Language Processing	自然语言处理
LM	Language Model	语言模型
CVE	Common Vulnerabilities & Exposures	通用漏洞披露
CWE	Common Weakness Enumeration	常见缺陷列表
NVD	U.S. National Vulnerability Database	美国国家漏洞数据库
GAP	Global Average Pooling	全局平均池化

第一章 绪论

源代码漏洞检测是软件安全领域中识别漏洞攻击和系统潜在威胁的重要保障。然而，由于现在软件的多样性和复杂性，用户在使用时会产生不同的情形，从而引起不同的漏洞问题产生，导致漏洞数量增加。同时，计算机软件的开发人员在编写代码时，会无意留下漏洞被攻击者利用。因此，针对智能化及高效性的源代码漏洞检测需求，本文聚焦于基于深度学习的源代码漏洞检测技术研究。本章将概述其研究背景及意义、课题来源及研究内容、章节安排。

1.1 研究背景及意义

随着计算机软件的发展和应用日益广泛，计算机系统使用量呈指数式增长，软件漏洞问题威胁着计算机系统的安全性。据统计，截至 2020 年，美国国家漏洞数据库（National Vulnerability Database, NVD）披露的安全漏洞记录数目达到 18325 条；截至 2022 年 10 月 10 日，公共漏洞和暴露数据库（Common Vulnerabilities and Exposures, CVE）中已存在 186011 个条目，漏洞数量呈爆炸式增长，所以探索并实现高效的软件漏洞存在性检验有重要的研究和应用价值。

因此，国内外学者对漏洞检测进行了大量的研究，常用漏洞检测技术根据分析对象主要分为两大类：基于源代码的漏洞检测和基于二进制代码的漏洞检测^[1]。基于源代码的漏洞检测通常采用静态分析方案。首先建立具体的漏洞检测规则，然后通过数据流分析、污点分析、符号执行等技术对相应的规则进行测试，从而实现漏洞检测。由于二进制代码通常是可执行的，基于二进制代码的漏洞检测方案可以分为静态、动态和动静组合。静态二进制分析方案需要在漏洞检测前将二进制代码转换成统一的中间语言再进行漏洞检测；动态二进制分析方案主要采用模糊测试技术；动静结合的分析方案通常是在动态测试时借助静态分析结果优化测试结果。由于面向二进制代码的漏洞检测存在表征工作复杂、数据集难寻和检测方法有限等缺陷，面向源代码的漏洞检测更具有研究价值。

基于源代码的软件漏洞检验问题是计算机系统的重要安全保障之一。近年来，由于机器学习和深度学习模型具有强大的特征表示学习和数据拟合能力，逐渐被应用在漏洞检测中，直接或间接地为漏洞检测提供服务。传统的机器学习方法中，数据预处理比分类器的选择更为重要，通常是使用文本特征提取的各类算法将源代码表征成机器学习可输入的形式，再使用支持向量机、随机森林等分类器进行训练，最后得出漏洞存在性检验结果。相比于机器学

习方法，基于深度学习的源代码漏洞检测方法能挖掘更深层次的漏洞信息，产生更好的检测效果。深度学习方法中，数据预处理方式更丰富，通常使用自然语言处理（Natural Language Processing，简称 NLP）方法获得漏洞源代码的表征，再将表征输入构建好的深度学习模型进行训练，在一定程度上挖掘漏洞的逻辑和语义信息，达到较高的漏洞存在性检验效率。因此，基于深度学习的源代码漏洞检测研究具有重要意义。

1.2 课题来源及研究内容

本文的研究内容主要来自导师的校企合作项目，该项目主要是设计一个漏洞检测系统。在该项目中，本人主要负责漏洞检测方法的设计与实现、后台基础业务的接口开发以及部分前端页面的开发，实际的系统开发也为本文的理论研究在应用层面提供了支撑。本文主要完成了以下内容：

（1）研究了漏洞检测的常用方法和漏洞特征表示形式，根据研究内容对基于深度学习的源代码漏洞检测方法进行了调研和学习。

（2）研究了组合模型对源代码漏洞进行检测的相关方法，并对相关方法进行改进，提出了一种基于 DistilBert-LSTM 与多项朴素贝叶斯的源代码漏洞检测方法 DBM-MNB，提高了漏洞检测的准确率。

（3）研究了特征融合对源代码进行漏洞检测的相关方法，学习了源代码的多种特征提取方式，提出了一种基于多特征融合的源代码漏洞检测方法 CHR-CodeBERT，尽可能获得完整的漏洞特征表示，降低了漏洞检测的误报率和漏报率。

（4）结合上述两种漏洞检测方法，实现了基于深度学习的源代码漏洞检测系统，并对重要模块进行测试，测试结果达到了预期的水平。

1.3 章节安排

本文主要研究了源代码漏洞检测系统的漏洞检测方法，在研究并实现漏洞检测方法的基础上，实现了基于深度学习的源代码漏洞检测系统。全文的章节安排如下：

第一章，绪论。阐述了本文的研究背景及意义，说明了课题来源，概括了研究内容。

第二章，相关技术研究。简述了源代码漏洞原理、静态漏洞检测技术和动态漏洞检测技术。针对本文所研究的基于深度学习的源代码漏洞检测方法，围绕基于神经网络的漏洞检测方法、基于 Transformer 的漏洞检测方法和基于深度学习表征的漏洞检测方法三方面对国内外研究现状展开叙述。

第三章，基于深度学习的源代码漏洞检测系统总体设计。首先对系统的需求分析进行阐述，设计了系统的总体架构，包括前端部分和后端部分，并对这两部分的流程进行解释；然后，对系统的功能模块进行了设计，包括文件上传模块、源代码预处理模块、漏洞检测模块和结果显示模块；最后对系统所用的数据库表进行了设计。

第四章，基于 DistilBert-LSTM 与多项朴素贝叶斯的源代码漏洞检测方法。首先进行了问题分析，对相关文献进行了研究，并对其中的相关方法进行改善，提出了基于 DistilBert-LSTM 与多项朴素贝叶斯的源代码漏洞检测方法 DBM-MNB，阐述了方法的总体架构，包括数据预处理、DBM 模型和 MNB 模型三部分。最后设计对比实验并对实验结果进行了具体的分析。

第五章，基于多特征融合的源代码漏洞检测方法。首先进行了问题分析，对相关方法进行了研究，总结了缺陷。对相关方法进行了改进，提出了基于多模型融合的源代码漏洞检测方法 CHR-CodeBERT，阐述了方法的总体架构，包括数据预处理、CNN 特征提取模块、HAN 特征提取模块、RNN 特征提取模块、基于加性注意力机制的特征融合模块以及 CodeBERT 分类模块。最后设计对比实验并对实验结果进行了详细的分析。

第六章，系统实现与测试。展示了系统的软硬件环境；阐述了系统方法的实现，包括后端检测模块和前端结果可视化模块；对系统的重要功能模块进行了测试，包括文件上传、源代码预处理、漏洞检测和结果显示。

第七章，总结与展望。总结了本文主要的研究工作以及研究成果，展望未来的研究工作以及研究方向。

第二章 相关技术研究

本章首先对相关基础知识进行阐述，对源代码漏洞产生的原理进行分类总结，然后阐述不同种类的漏洞检测方法。接着结合国内外文献，对基于神经网络的漏洞检测方法、基于 Transformer 的漏洞检测方法以及基于深度学习表征的漏洞检测方法进行研究概述。分析以上方法的研究现状，最后对本章节的内容进行了总结。

2.1 基础知识

2.1.1 源代码漏洞原理介绍

现如今，软件的功能越来越多，系统越来越复杂，开发人员将开源代码引入到项目中能够极大地提高开发效率；但在增加开发便利度的同时，引入开源代码也增加了软件系统存在漏洞的可能^[2]。源代码漏洞会造成危害软件安全的潜在危险，严重的漏洞将允许黑客通过附加端点利用漏洞提取数据、篡改软件程序，了解漏洞产生的原理有助于预防其带来的危害。源代码的漏洞分类方式通常与漏洞的产生原理相关，为此以下四种漏洞类型是有必要了解的。

（1）注入漏洞：注入漏洞允许攻击者通过简单的系统调用将代码“注入”到系统中。这些调用通常是通过 shell 命令使用外部程序完成的。对数据库或 SQL 的注入^[3]是最常见和最危险的。通常，攻击者找到一个通过数据库传递的参数，利用该参数携带一个恶意的 SQL 命令作为内容。数据库将其存储并将其误认为代码，从而欺骗软件发送、更改或删除数据。SQL 注入非常危险，几乎没有任何 Web 应用程序是安全的，因为它们均在外部命令上运行。所有人能做的就是，将程序编码尽可能做到完美，以至于注入更加复杂，从而不会留下任何漏洞。

（2）跨站点脚本（Cross Site Scripting, XSS）^[4]：它是一种注入，但由于受害者是网站，将其归类为独立注入。XSS，意为跨网站脚本攻击，是在网站上完成的。通常将 Javascript 和 HTML 中的恶意脚本作为数据注入到站点，在站点中它可以附加自身以引起安全问题。用户浏览器几乎不可能检测到此类脚本，因为对浏览器而言，脚本来自受信任的来源。它通常由包含敏感信息的代码完成，例如联系电话或信用卡详细信息等。

（3）缓冲区溢出：缓冲区是分配给包含字符串或整数等数据的顺序内存。考虑该缓冲区是否被数据或请求轰炸超过了它可以处理的范围，它会溢出到相邻的存储空间中。这种溢出可能会造成重大问题，例如软件崩溃、数据丢失或最危险的问题——为网络攻击创建入口点。

此代码漏洞称为缓冲区溢出^[5]，取决于不同的编程语言。Javascript 和 Pearl 是避免此类攻击的两种语言，但构建块语言 C 和 C++ 会受到深刻影响，整个系统可能会受到损害。攻击者通常通过覆盖软件中执行路径的代码块来做到这一点，数据可能包含一些脚本或代码，这些脚本或代码可能会提示软件执行某些不需要的活动。

(4) 破碎的身份验证 (Broken Authentication, BA)^[6]：在 OWASP Top 10 的顶级代码漏洞图表中排名第二，BA 这些年来甚至困扰着顶级产品和网站。当攻击者使用不同的方式进入其他人的帐户时，就会出现身份验证失败的漏洞。它会导致错误授权，然后再次丢失敏感数据。虽然其中一些问题不是开发人员的错，但构建可以解决此类问题的健壮代码仍然是编码人员的责任。在没有多重验证或会话超时的情况下，代码变得容易受到攻击。Web 应用程序中最常见的代码漏洞是：当为用户创建会话 ID 并且黑客以某种方式检索并使用地址重写来重新创建该会话。另一种方法是，如果黑客可以使用其他安全漏洞进入用户的密码数据库，并且密码没有正确散列和加密，那么黑客可以反转编码并显示每个人的密码。

2.1.2 静态漏洞检测技术

静态分析是指在不运行软件的情况下，通过构建抽象语法树、依赖图、序列等来分析程序的过程，其分析对象通常是源代码或可执行代码。与可执行代码相比，源代码分析可以获得更多的语义信息，综合考虑执行路径上的信息，从而发现更多的漏洞，提高命中率。目前，静态漏洞检测技术通常由搭建的分析工具完成，主要有：

(1) VulDeeLocator^[7]：一种可以同时实现高检测能力和高定位精度的漏洞检测器，称为基于深度学习的漏洞定位器 (VulDeeLocator)。利用中间代码来容纳额外的语义信息，使用粒度细化的概念来确定漏洞的位置。

(2) Flawfinder^[8]：一种静态分析器，用于检测 C/C++ 语言编写的源代码漏洞。Flawfinder 与常见缺陷列表兼容，C/C++ 程序作为输入提供给该工具，然后生成按风险级别排序的命中（漏洞）列表。Flawfinder 报告的每个漏洞都分配了一个从 0 到 5 的风险级别，0 表示漏洞被利用的风险最低。

(3) RATS^[9]：Rough Auditing Tool for Security 的缩写，最初是审计 C/C++ 源代码安全漏洞的静态分析工具，后来开发到可扫描各种语言，包括 C、C++、Perl、PHP 和 Python。该工具可检测到常见的安全问题，如缓冲区溢出、TOCTOU（检查时间、使用时间）等。

(4) CPPCheck^[10]：一种 C/C++ 源码漏洞检测的静态分析器，它可以检测任何语法错误。该工具专注于没有漏报，这意味着该工具不会报告任何 wrong errors，它报告的错误数量少于

实际上存在于源代码中的数量,结果导致许多假阴性。CPPCheck 强烈关注检测未定义的行为,如死指针、除以零、整数溢出和空指针取消引用等。

(5) SPOTBUGS^[11]: 一个静态分析器,可以针对不同类别的错误审计 Java 源代码。它可以检查超过 400 个错误模式,是 FINDBUGS 静态分析工具的继承者。SPOTBUGS 报告不同类别的错误,它附带一个用于检测安全性的插件,在 Java 源代码中称为 FIND_SEC_BUGS (查找安全漏洞)。

(6) PMD^[12]: 一种用于静态分析 Java 源代码以找出各种编程缺陷的工具,例如未使用的变量为空的 catch 块,不必要的对象创建等。PMD 具有许多内置检查功能,它支持专用 API 编写用户的规则,可以在 Java 中作为自包含的 XPath 完成询问。它可以与各种平台集成,如 JDeveloper、Eclipse、JEdit、JBuilder、Blue J 等。

2.1.3 动态漏洞检测技术

动态分析是通过运行特定程序并获取程序的输出或内部状态等信息来验证或发现软件漏洞的过程,其分析对象是可执行代码。动态分析方法一般应用于软件的测试运行阶段,在软件程序运行过程中,通过分析动态调试器中程序的状态、执行路径等信息来发现漏洞^[13]。与静态方法相比,动态方法通过分析漏洞获取具体的运行信息,因此分析出的漏洞一般更准确,误报率更低。

文献[14]提出了一种基于深度学习的动态分析软件漏洞检测方法。通过二进制执行、事件挂钩、事件收集三个步骤和 VDiscover 工具^[15]确定最终的执行状态。然后,使用 zzuf 工具实现数据标注。

文献[16]从智能种子的生成和选择开始,以减少对无用路径的探索,提出的解决方案是 NeuFuzz,它使用深度神经网络从大量易受攻击且干净的程序执行路径中学习隐藏的漏洞模式。在线引导模糊测试时,利用预测模型判断未见过的路径是否存在漏洞,根据预测结果标记种子,加入种子队列。最后,在接下来的选种过程和种子变异过程中,弱势种子将被优先考虑并分配更多的变异能量。

由于缺乏专门的程序分析工具,尤其是动态分析工具,Avatar^[17]提出了一种动态多目标编排框架,旨在实现不同动态二进制分析框架、调试器之间的互操作性,以及仿真器和真实的物理设备之间的操作转发。它允许分析者以复杂的拓扑结构组织不同的工具,然后从一个系统移动到另一个系统执行二进制代码,有效解决了特定外围元器件没有源码和文档的问题。随后,Prospect^[18]和 Surrogate^[19]也提出了类似的动态分析框架。2018 年,Avatar 的作者团队

继续推出了 Avatar2^[20]，它允许分析者在不同的动态二进制分析框架、调试器、仿真器和真实的物理设备之间进行互操作。文献[21]提出了 Firmadyne，专注基于 Linux 的设备，先用软件进行全系统模拟，然后采用扫描、探测等动态分析方法发现漏洞。上述框架的仿真特性均基于 QEMU^[22]。

2.2 基于神经网络的漏洞检测方法

得益于深度神经网络在图像识别、自然语言处理等任务上取得的巨大成功，人们发现，深度学习方法在挖掘漏洞深层特征上更加具有优势^[23]。图 2.1 展示了代码静态分析和神经网络训练的原理。整个过程包括以下步骤：样本代码构建、特征提取、词向量生成、神经网络模型训练与分类。其中，漏洞特征提取主要涉及如何选择合适的粒度来表示软件程序和漏洞检测。由于源代码需要转换成向量才能输入神经网络，因此需要将其表示成具有一定语义意义的向量来进行漏洞检测。使用一些中间表示作为程序与其向量表示之间的“桥梁”，这是深度学习的实际输入。漏洞特征提取是通过控制流图和过程依赖图等技术，将程序转化为某种中间表示，可以保留程序元素之间的（部分）语义关系（如数据依赖和控制依赖）。词向量生成是在特征提取的基础上，应用最主流的词向量生成技术，将中间表示转化为向量表示，即神经网络的实际输入。神经网络训练分类涉及训练和检测两个阶段。训练阶段以从历史代码库中提取的代码向量表示作为输入，其输出是微调模型参数的神经网络。在检测阶段，将从新软件程序中提取的代码向量表示作为输入，输出为分类结果。

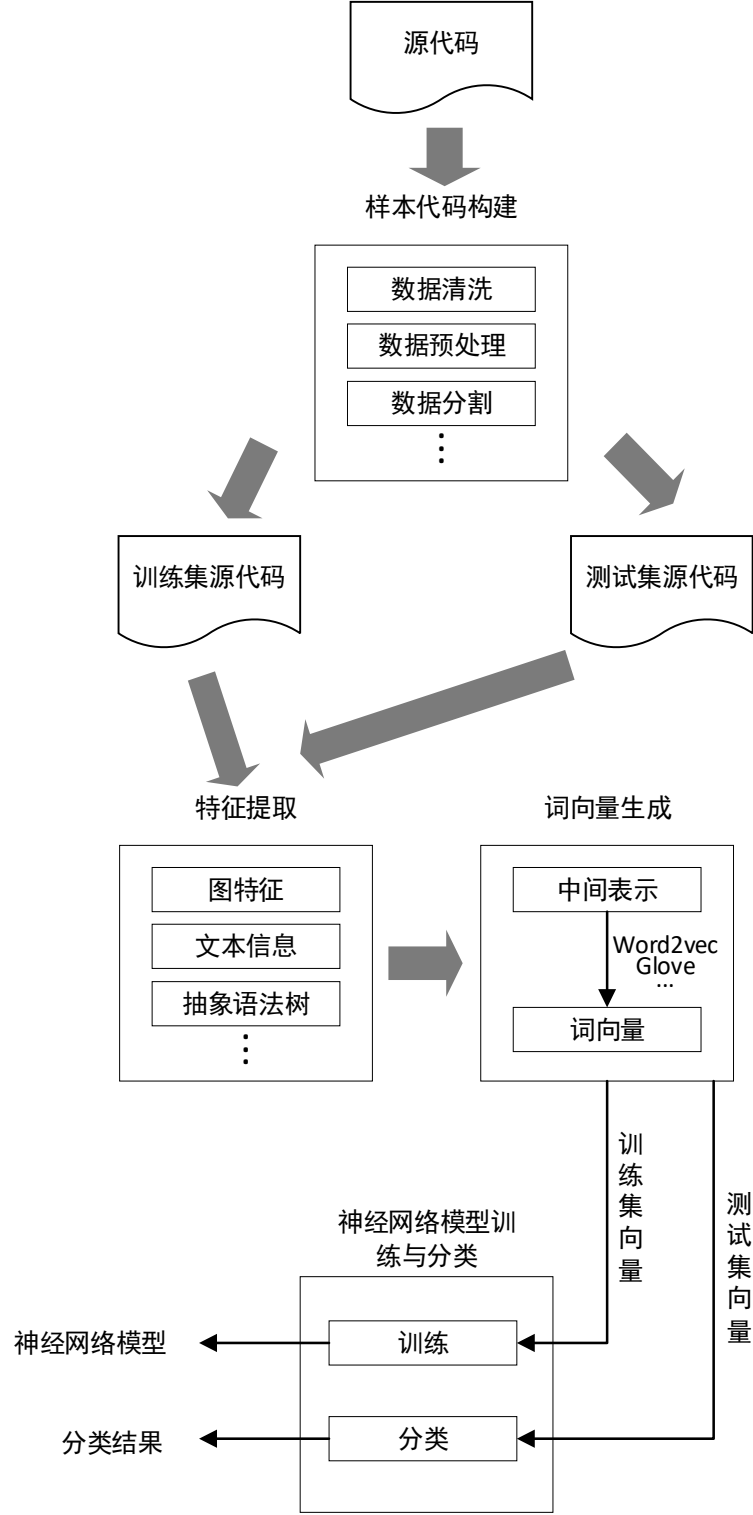


图 2.1 代码静态分析和神经网络训练流程图

肖添明等人^[24]基于代码属性图和 Bi-GRU 对软件脆弱性进行检测，使用代码属性图构建源代码的漏洞特征，通过 Bi-GRU 进行特征提取，最后对多个软件源代码数据集进行分类检测，实验结果表明此方法有效降低了漏洞检验的误报率和漏报率。文献[25]提出一种基于关系图卷积网络（RGCN）的漏洞检测方法，该方法通过程序依赖图提取图表示信息，再用 RGCN 模型进行预测，进一步提高漏洞检测的准确性能。文献[26]提出了一种结合卷积神经网络和高

速神经网络的源代码漏洞检测模型，通过两种神经网络捕获更多的漏洞特征，达到漏洞检测的效果。

文献[27]提出了一种基于句子的深度学习代码漏洞检测方法并定义了八种 token 类型。首先，使用 ANTLR^[28]解析每个函数并提取 C1-C3 三种 token，并使用 Eclipse ASTParser^[29]创建每个函数的抽象语法树进一步提取 C4-C8 五种 token。计算每种 token 在每种函数中出现的频率以及源代码函数和漏洞函数之间的相似度，输入到神经网络分类器进行训练以检测给定代码库的漏洞。由于 token 仅显示语法层面的代码关系，这个能力衡量源代码函数与漏洞代码函数之间相似度的方法是有限的，漏洞检测效果也有待提高。

文献[30]提出了一种新颖的代码漏洞检测方法，将二进制代码转换为图像数据输入到训练模型的卷积神经网络。同时，作者使用 Tigress C 混淆器^[31]迭代提取原始数据集扩展的转换语法充分训练模型，提升漏洞检测的准确率。由于卷积神经网络需要查看整个图像，从二进制代码文件中提取完成每个文件的分类，因此二进制代码的规模限制了系统的性能。在实验阶段，卷积神经网络中的所有权重都是从标准偏差设置为 0.1 的正态分布中获取的随机值进行初始化，以避免运行在早期进入局部最小值，并进一步微调 test_ratio、gradient_rate、limit 和 random_seed 等参数以提高漏洞检测的性能。

为了证明组合神经网络模型在检测多种代码表征方法上的有效性，文献^[32]实现了四种代码表示：标识符、抽象语法树、控制流图和字节码，并为每个表示训练神经网络模型，通过整合不同的模型，代码漏洞检测的准确率得到进一步改善。

深度学习技术可以减少人类特征工程，有望在未来的研究过程中取代传统的漏洞检测方法，提高漏洞检测性能。然而，深度学习在漏洞检测方法上存在诸多不足，神经网络模型往往是基于程序源代码进行训练的，这在大多数情况下是不可用的。此外，检测粒度和有限的神经网络进一步限制了检测效果。

为了进一步推进自动化漏洞检测技术的发展，在未来的研究过程中，可以考虑对具有相似语法和语法的编程语言进行抽象建模，以实现具有相同神经网络模型的多种语言之间的软件漏洞检测。同时可以考虑通过迁移学习来实现小块数据的学习过程，以解决缺乏大规模漏洞数据集的问题。此外，神经网络结构的优化和特征参数的选取一直是相关从业者面临的问题^[33]。现有的代码漏洞检测技术往往基于 10 折交叉验证训练方法来获取最优参数，这种方式对于大型项目数据往往会牺牲训练时间和模型性能。

2.3 基于 Transformer 的漏洞检测方法

基于 Transformer^[34]的大型语言模型在自然语言处理方面表现出色。考虑到语言模型在一个领域中获得的知识到其他相关领域的可迁移性,以及自然语言与高级编程语言(如 C/C++)的接近性,如何利用基于 Transformer 的语言模型来检测软件漏洞是一大研究热点。

使用基于 Transformer 的软件漏洞检测模型具有以下优势:(1)它使用静态分析工具进行不可能的自动化检测,使用启发式方法查找已知漏洞的代码构造,不需要大量的手动操作;(2)它消除了泛化的特征工程需求,如传统的机器学习。

尽管最近在漏洞检测中使用了一种基于 Transformer 的语言模型 BERT (Bidirectional Encoder Representations from Transformers)^[35],但目前尚不清楚其他方法如何使用模型,例如生成式预训练变压器(GPT)^[36]。此外,训练/微调这些模型总是受到计算要求、参数和依赖项等限制。

为使程序文件的生成自动化,方便自然语言代码搜索,一种基于 Transformer 的语言模型 CodeBERT^[37],接受成对的自然语言和编程语言训练。例如,一个源代码片段有关于程序描述的自然语言,或者是代码说明。CodeBERT 的架构包括一个 BERT 模型和两个生成器,一个用于代码指令,另一个用于自然语言描述。CodeBERT 有几个变体,例如,使用语义级的 GraphCodeBERT^[38]数据流结构、BART^[39]、PLBART^[40]以及带有编码器和解码器模型架构的一部分。

BERTBase 模型用于软件漏洞检测^[41]。它已通过软件保障参考进行了微调,包含 100KC/C++源文件的数据集(SARD)数据库并经过测试 123 个漏洞。这项工作表明 BERTBase 模型和具有 LSTM 或 BiLSTM 模型的 RNN 头的 BERT 表现优于标准 LSTM 或 BiLSTM 模型,最高的检测精度为 93.49%。与此相反,考虑一个大数据集和多个模型架构(如 GPT-2 和 MegatronBERT^[42])对漏洞检测的效果并没有更优。

2.4 基于深度学习表征的漏洞检测方法

源代码错综复杂,通常需要转换成深度学习可以识别的形式进行学习,对其有效的表征才能更好的挖掘漏洞特征。为此,国内外研究学者展开了大量的研究,对于挖掘深层次地源代码漏洞信息,代码表征方式主要可以分为 5 类,分别是:基于代码度量的表征方式、基于抽象语法树的表征方式、基于图的表征方式、基于序列的表征方式、基于文本信息的表征方式^[43]。本文主要对以下几种表征方式进行介绍:

代码度量：代码度量是一组用于衡量软件质量的软件度量值。基于代码度量的表示方法选择多个代码度量指标，得到一个能够概括代码总体信息的度量序列，并用该度量序列表示相应的代码^[13]。文献[44]、[45]、[46]调查了软件从源代码和开发历史中获得的指标是否对易受攻击的代码位置具有辨别力和预测性。Shin 等人^[47]考察了适用三种类型的软件指标（复杂性、代码改动以及开发人员活动）来构建漏洞预测模型。作者对两个开源项目进行了实证分析项目：Mozilla Firefox 和 Red Hat EnterpriseLinux kernel，发现收集到的 28 个指标中有 24 个是对这两个项目的脆弱性进行区分。其他工作^[48]证明了一些微不足道的软件指标如字符多样性、字符串熵、函数长度和嵌套深度可能是有用的漏洞检测指标。

抽象语法树和图：代码矢量化是基于深度学习的代码审计的先行研究。抽象语法树（Abstract Syntax Tree, AST）和控制流图（Control Flow Graph, CFG）是最流行的技术和经典的代码分析方法。它们抽象出代码逻辑，给出序列化结果，相当适用于深度学习模型的输入。基于 AST 的代码向量化方法最早是由 Michael 等人提出的^[49]。文献[50]中这两个特殊的数据结构代表源代码，消除程序员不同代码风格带来的噪音。然而，缺点是它们都将原来的代码结构复杂化为另一种数据结构。新的复杂的数据结构需要额外的方法来解析，这无疑会增加研究人员的工作量。在某种程度上，它可能会由于不同的原因而丢失一些重要的信息解析方法。基于这种方法，RIPS^[51]、Cobra^[52]等审计工具提出了基于代码的上下文检测方式。RIPS 专注于敏感函数的调用、数据的跟踪以及支流的分析。Cobra 基于词法分析将源代码解析为 AST，然后判断函数代码段是否存在漏洞。然而，这些主流工具误报率高，即误报相对安全的代码块作为漏洞。

文献[53]提出了一种基于图神经网络的代码漏洞检测方法，通过中间语言的控制流图特征，实现函数级别的智能化代码漏洞检测。文献[54]从源代码程序依赖图中根据漏洞特征提取图结构源代码切片，将图结构切片信息表征后利用图神经网络模型进行漏洞检测工作。实现了切片级的漏洞检测，并在代码行级预测漏洞行位置。文献[55]提出了一种基于代码属性图及注意力双向 LSTM 的源码级漏洞检测方法，该方法首先将程序源代码转换为包含语义特征信息的代码属性图，使用基于双向 LSTM 和注意力机制的深度学习模型实现对目标程序中的漏洞进行挖掘。

序列：随着自然语言处理（NLP）的发展，代码的标记化时序分析成为审计的新方向。受到 NLP 相关技术的启发，标记器将源代码解析为标记序列。一个优点是序列保留源代码的强上下文语义。第二个是它清除了数据噪音，这是编码人员的个人习惯造成的，例如因变量名称和缩进。同时，序列化的源代码可以使用向量的语言模型（LM）生成空间嵌入。随着神经网络结构和算法与时间序列的结合长短期记忆（LSTM）等特性，该模型可以取得良好的性能

[56]。此外，序列化方法避免引入额外和更复杂的数据结构，这种方法目前比较合理，值得进一步研究。Fang 等人提出的 TAP tokenizer^[57]是序列化 PHP 源代码的有效方法，其基于自定义规则完成对 CWE 各种漏洞的审计数据集。在他们的测试中，TAP 方法达到了 0.9787 的最高精度和 0.9941 的曲线下面积。但是，漏洞代码块的召回率仍然有待提高，这是一个普遍的问题，也是同一类别的其他研究中存在的问题。在 Liu 等人的工作中，提出了一个跨域软件漏洞系统使用深度学习和域适应的发现（CD-VulD）^[58]。作者通过 AST 工具将跨域程序表示为 token 序列，然后使用 token 序列构建用于漏洞检测的分类器。实验结果表明，CD-VulD 的召回率优于最先进的漏洞检测方法。

文本信息：据文献[59]称，预测技术，在智能软件分析和基于文本挖掘的性能优于基于代码度量的预测技术。这种方法将源代码视为常规文本并利用自然语言处理（NLP）技术进行代码表示和特征提取。虽然从理论上讲，编程语言是复杂、灵活和强大的，但真人实际编写的代码片段大多是简单且重复的，因此这种方法很有用。将源代码表示为文本能捕获可预测统计属性并用于软件漏洞检测任务^[60]。例如，根据 NLP 中的词嵌入概念，文献[61]的作者生成了一个集合，包含经过大量预训练的通用模型代码，这些代码可用于漏洞检测时的正向数据集。另一个值得注意的研究^[62]采用基于 NLP 的处理方法进行漏洞检测，这种方法对“易受攻击”和“不易受攻击”的源代码进行分类，模型通过独热编码学习深度特征表示，从词法分析的源代码中获得标签。为了进行性能评估，作者建立了自己的数据集（Draper VDISC Dataset）实现基于文本信息表征的漏洞检测。

2.5 本章小结

本章首先对本文中相关概念的背景知识以及内容进行了介绍，分析了源代码漏洞原理、静态漏洞检测技术和动态漏洞检测技术。接着对国内外基于神经网络的漏洞检测方法、基于 Transformer 的漏洞检测方法以及基于深度学习表征的漏洞检测方法进行了详细概述，通过对上述文献的分析发现，现有的漏洞检测方法主要存在以下问题：

（1）现有的漏洞检测方法通常基于单模型对源代码进行检测，忽略了组合模型对漏洞检测的深度，例如模型的层次数量会影响漏洞检测的准确率，造成时间维度或者空间维度的信息丢失。将多模型融合，一方面降低单模型对漏洞检测带来的漏报率和误报率，另一方面将模型的多层正面效果叠加，充分考虑漏洞检测的时空特征。如何将多个模型的高效性叠加带来更高的漏洞检测准确率，成为目前亟待解决的问题。

（2）源代码的深度学习表征形式直接影响到漏洞检测的效率，为了实现特征表示的可视

化，目前基于图和树的分析方法十分流行，例如控制流图、数据依赖图和抽象语法树等。虽然图和树简洁了代码的表示，但同时也降低了漏洞特征的完整度，不必要的分支也增加了特征挖掘的阻力。如何从原始的代码中深度挖掘漏洞特征信息，成为目前亟待解决的问题。

综上所述，针对以上所提及的问题，本文采用组合模型和多特征融合的漏洞检测方法，针对漏洞数据库的数据，本文共提出两种不同的源代码漏洞检测方法，从不同层面提高漏洞检测的准确率，具体的检测方法将在后面的章节中进行详细阐述。

第三章 基于深度学习的源代码漏洞检测系统设计

本章节对基于深度学习的源代码漏洞检测系统进行了总体设计。首先对系统的需求分析进行阐述，设计了系统的总体架构，包括前端部分和后端部分，并对这两部分的流程进行解释；然后，对系统功能模块进行了设计，包括文件上传模块、源代码预处理模块、漏洞检测模块和结果显示模块；最后对系统所用的数据库表进行了设计。

3.1 系统总体设计

3.1.1 系统需求分析

本论文所设计的基于深度学习的源代码漏洞检测系统，主要是为了实现软件源代码的漏洞检测功能。在系统的应用层面上，系统主要服务对象有平台用户以及后台运营人员、测试人员。普通用户直接将待检测的源代码文件保存为系统对应格式（后缀为.json/.zip），点击上传按钮对源代码进行漏洞检测，页面显示检测结果，包括：检测时间、检测文件数量、是否存在漏洞和结果统计图。对于后台运营人员，需要维持服务器系统的稳定，能更新后台漏洞检测模型和结果可视化的展示形式。测试人员借助后台的测试系统对相关功能进行测试，对问题进行收集、反馈。

3.1.2 系统总体架构

本系统开发的源代码漏洞检测系统共涉及四个功能模块，其中前端部分两个模块，后端部分两个模块，并用 Django 框架整合前后端，系统架构如图 3.1 所示。

前端部分包括文件上传模块与结果显示模块。其中，文件上传模块主要包括文件格式的判断，目前支持.json 和.zip。文件的上传与检测可以同时选择多个文件进行上传。压缩文件上传支持将大量要检测的源代码文件压缩成一个压缩文件然后进行上传。无论是单文件上传还是压缩文件上传，其上传结果都会在结果显示模块进行显示。同时会将结果保存在结果储存部分，并且可以通过结果导出部分将检测结果进行导出。

后端部分包括源代码预处理模块与漏洞检测模块。其中，源代码预处理模块主要是将上传的代码进行清洗操作，去除多余的字符，统一数据格式。漏洞检测模块先将代码进行特征向量化表示，再将向量输入到第四章、第五章建立的模型中进行分类，具体模型将在后面的

内容进行介绍。

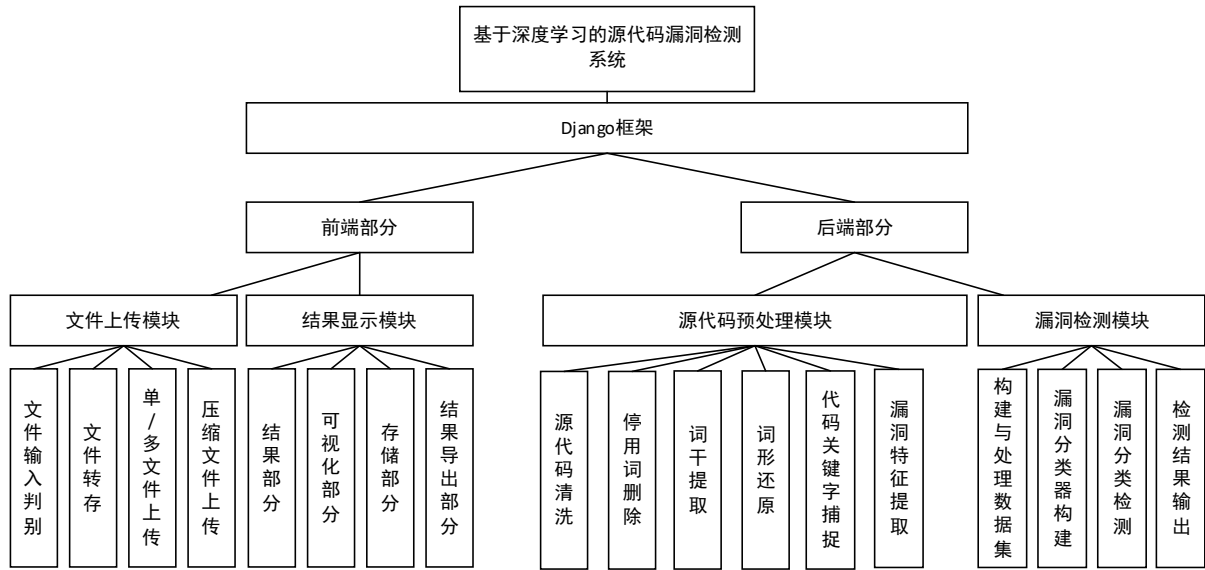


图 3.1 基于深度学习的源代码漏洞检测系统架构图

3.2 系统前后端设计

3.2.1 前后端框架与请求/响应设计

本系统采用 Django 框架，其应用于 Web 服务器开发领域著名的 MVC 模式，即将 Web 应用分为模型（M）、控制器（C）以及视图（V）层，层间通过插件式与松耦式方式连接，模型负责业务对象与数据库的映射（ORM），视图负责与用户的交互（页面），控制器接受用户的输入调用模型与视图完成用户的请求，其示意图如图 3.2 所示。

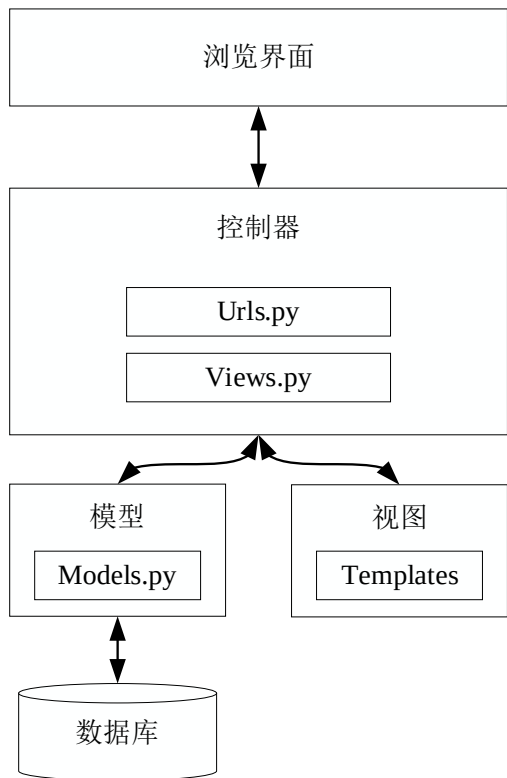


图 3.2 漏洞检测系统框架示意图

其中，请求响应架构如图 3.3 所示：

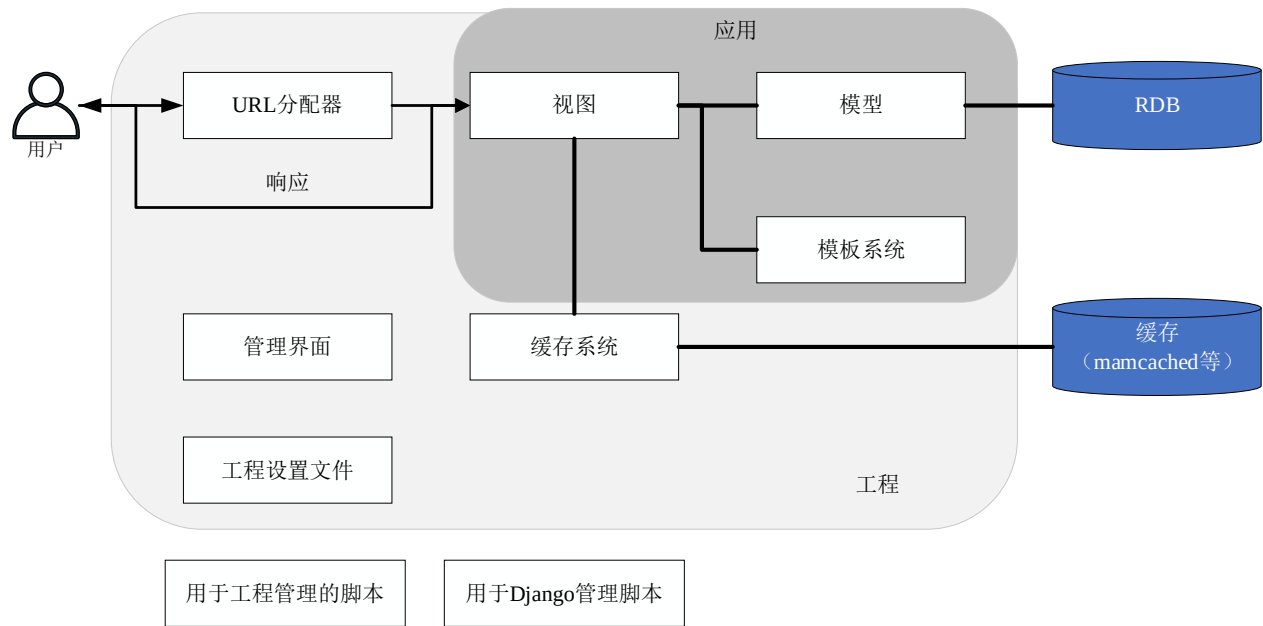


图 3.3 漏洞检测系统请求响应架构示意图

- 1. 客户端发来的 HTTP 请求被视为 Django 的请求对象；
- 2. URL 分配器负责搜索并调用被请求的 URL 所对应的视图；
- 3. 被调用的视图根据情况使用模型或模板生成相应的对象；
- 4. 相应对象作为 HTTP 响应发回给用户。

3.2.2 系统前端流程

本系统前端分为源代码文件上传、文件上传和结果显示三个部分，流程如图 3.4 所示。

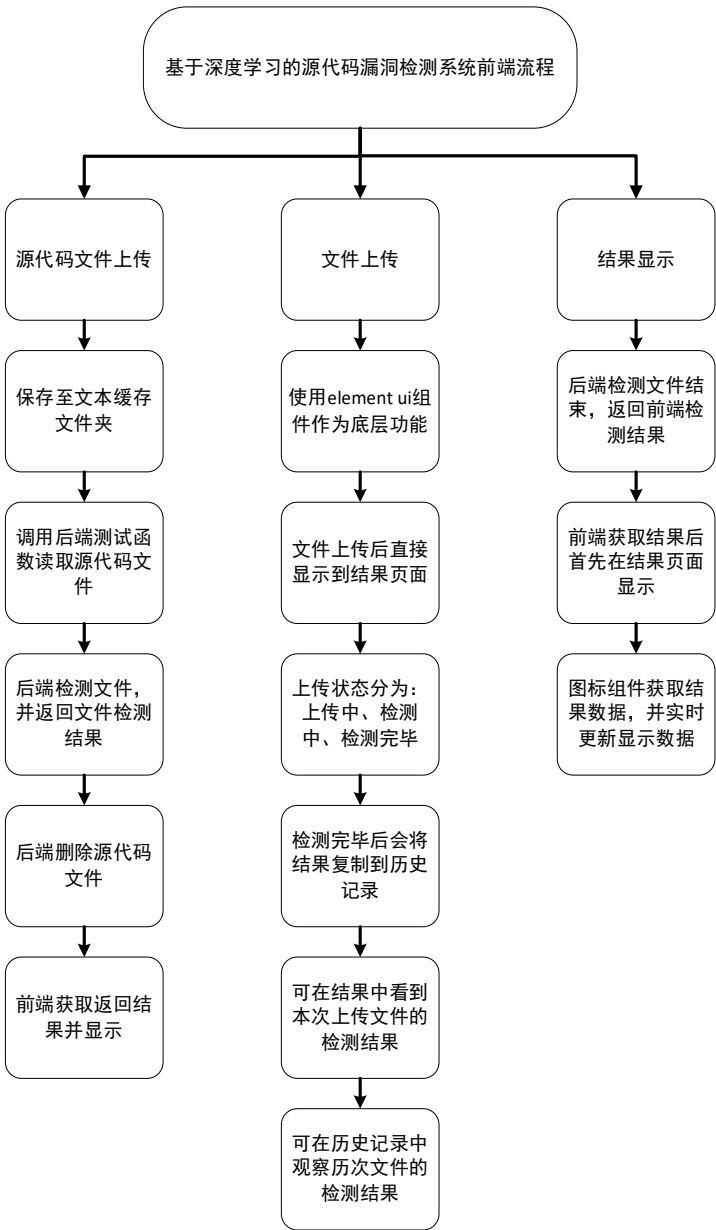


图 3.4 漏洞检测系统前端流程示意图

- (1) 源代码文件上传：首先将文件保存至文本缓存文件夹，然后调用后端测试函数读取代码文件，读取完成后继续检测文件，并返回检测结果给前端，检测结束删除代码文件。
- (2) 文件上传：使用 element ui 组件作为底层功能，文件上传后直接显示到结果页面，共有四种上传状态，分别是上传中、检测中、检测错误和检测完毕。当文件不是源代码规定的上传文件时会显示为检测错误。检测完毕后端返回检测结果后会将结果复制到历史记录中，可在历史记录中查看历次上传代码的检测结果。
- (3) 结果显示：结果显示页面分别有结果统计、图表统计、具体检测信息、历史检测信

3.2.3 系统后端流程

本系统后端分为源代码输入、文本特征提取、模型检测和输出结果四个部分，流程如图 3.5 所示。

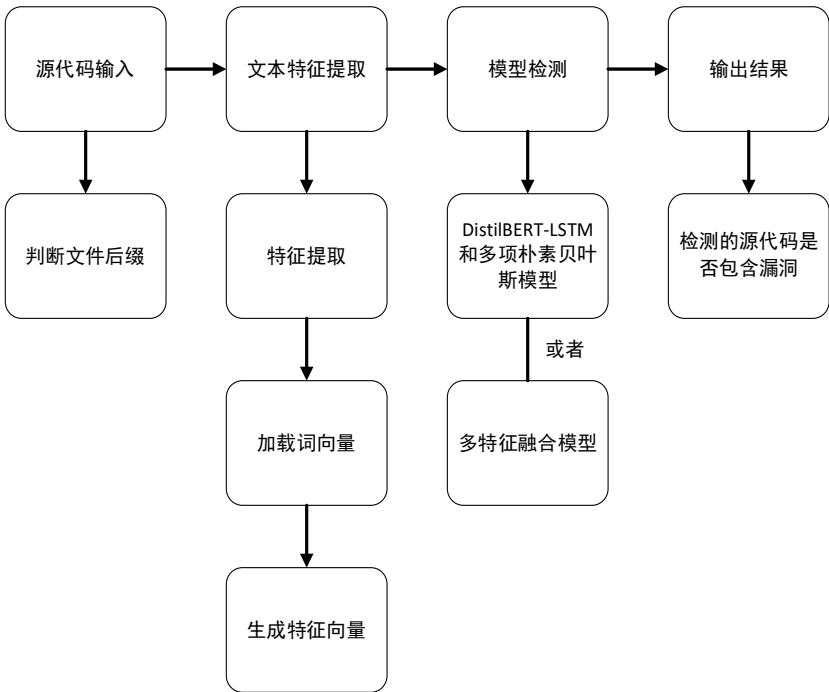


图 3.5 漏洞检测系统后端流程示意图

首先检测前端上传的源代码文件后缀是否是规定的格式，符合文件缀名后，后端的源代码处理函数会对代码进行清洗操作，读取包含源代码的列名称，列出要删除的停用词列表，将源代码的文本全部转换为小写并删除标点和字符，然后将清洗后的文本剥离出来，把源代码从字符串转换为列表。根据制作好的停用词包删除源代码中全部的停用词，从列表返回清洗完毕的代码。最后将源代码输入搭建的模型进行检测，返回漏洞分类的结果。

- （1）源代码输入：前端传入源代码文件之后对于传入文件后缀进行判断，看是否为.json或.zip，文件后缀如果是这两种就继续进行检测，如果不是则需要重新上传正确的文件。
- （2）文本特征提取：1）根据检测模型提取相应的特征，包括 TF-IDF 特征、CNN 特征、HAN 特征、RNN 特征；2）加载词向量：使用 Word2Vec 和 Glove 方法构建每个特征的特征矩阵；3）对源代码提取两个向量[Ids, Masks]和[Ids, Masks, Segments]。
- （3）模型检测：使用的检测模型分别是 DistilBERT-LSTM 和多项朴素贝叶斯模型（DBM-MNB）、多特征融合模型（CHR-CodeBERT）。
- （4）输出结果：通过以上模型任意之一检测得出分类结果，分类结果为当前检测的源代

码是否存在漏洞。

3.3 系统功能模块设计

本系统共包含 4 个功能模块，其中，前端部分包含文件上传以及结果显示 2 个模块，后端部分包含源代码预处理及漏洞检测 2 个模块。系统的功能模块图如图 3.6 所示，UML 时序图如图 3.7 所示。

文件上传模块完成单文件上传、多文件上传和压缩文件上传功能，支持文件解压、多格式源代码文件类别识别，当用户上传非源代码文件时，系统会显示文件识别错误，并支持重新上传。

结果显示模块通过后端返回的实时数据，动态显示检测结果，包括检测结果统计、检测结果可视化、上传状态可视化、历史检测结果统计功能。通过检测结果数据可视化分析，可直观获取漏洞检测结果，同时可实现历史检测结果查询功能。

源代码预处理模块通过去除无用信息对数据进行清洗，删除源代码中的空行、表情、标点和注释等，防止这些无用信息被表征，增加模型的输入维度。

漏洞检测模块基于 DistilBert-LSTM 和多项朴素贝叶斯、多特征融合两个模型，对清洗的源代码数据进行特征工程，表征成特征向量，输入到训练好的学习模型进行检测。本系统使用漏洞数据库的源代码进行检测，检测结果为输入的源代码是否存在漏洞。



图 3.6 漏洞检测系统功能模块图

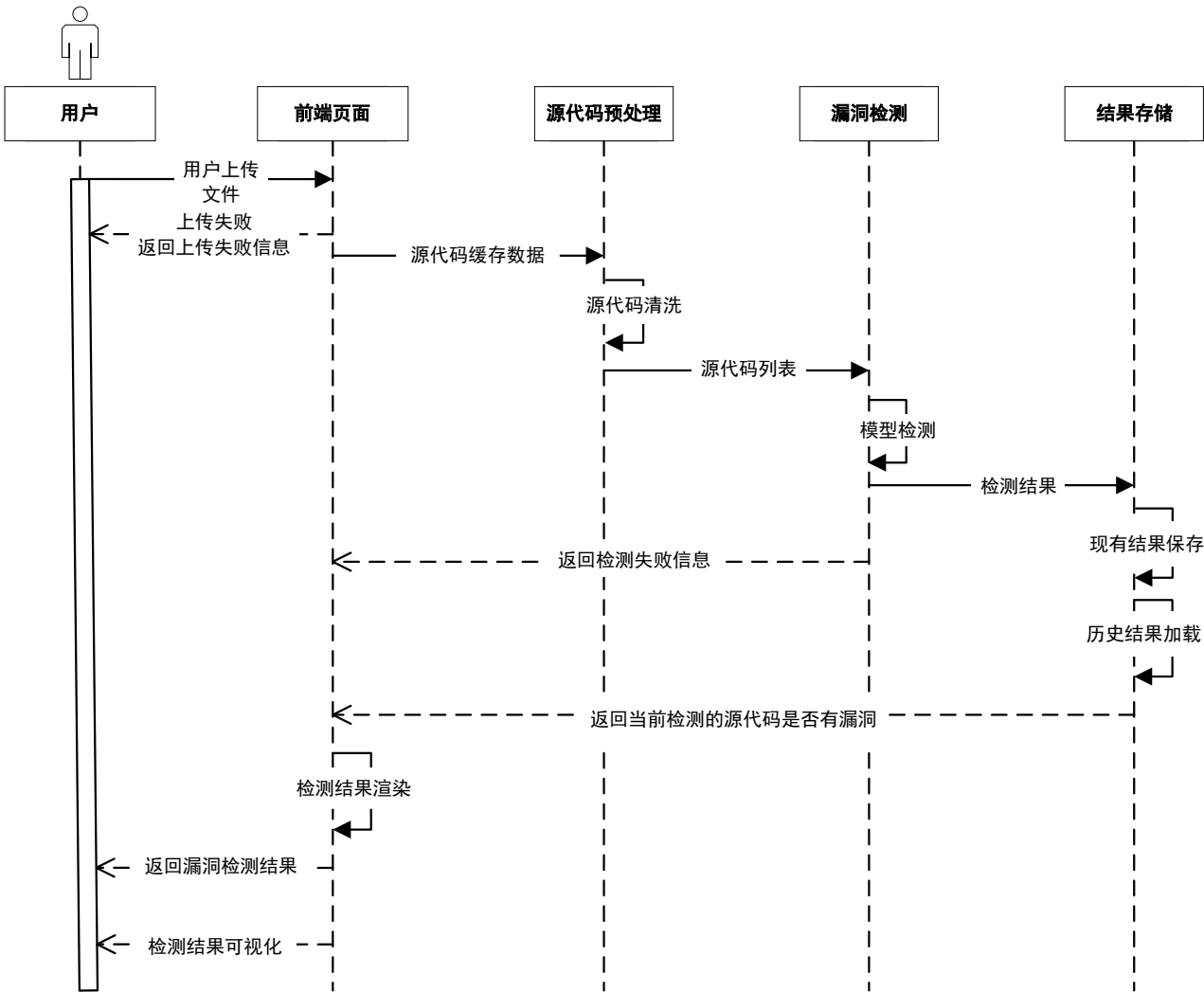


图 3.7 漏洞检测系统 UML 时序图

3.3.1 文件上传模块

文件上传模块分为两个上传功能，一个是批量上传多个文件，一个是上传压缩文件。用户可以通过点击上传或者上传文件夹按钮来将自己本地的文本文件上传到服务器检测。

（1）用户可选择上传单个源代码文件或多个源代码文件，确定上传后，文件具体信息会自动在结果显示模块的具体检测信息中显示出来。

（2）用户可选择压缩文件上传，系统会自动将压缩文件内的文件进行上传。当文件夹内有非文本文件时会显示检测错误。同时，上传的压缩文件中不能存在任何其它类型的文件和文件夹，否则会显示检测错误。

3.3.2 源代码预处理模块

根据前端上传的源代码文件，获取源代码的文本。由于代码中包含许多无用信息，会影响系统检测的结果，因此需要对源代码进行相关预处理操作。首先，对函数源代码进行清洗，删除源代码中的空行和注释等，防止这些无用信息被表征，增加模型的复杂度；然后，将清洗后的代码保存在列表中；最后，将预处理后的代码输入到模型进行训练。

3.3.3 漏洞检测模块

对预处理后的源代码数据进行特征工程，提取漏洞的特征信息，构建模型需要的特征矩阵，经过特征选择、降维、最大平均池化等操作形成可以直接输入到模型的向量，调用漏洞检测模型对源代码数据进行检测，检测结果为标签“1”或“0”，标签“1”则表示检测的源代码有漏洞，标签“0”则表示检测的源代码无漏洞，将结果传送至前端进行可视化。

3.3.4 结果显示模块

结果显示模块包含五个功能，分别以不同的方式直观展现上传源代码文件的检测结果，其中包括结果统计、图表统计、具体检测信息、历史检测信息以及导出模块。

（1） 结果统计

用于展示上传文件及其检测的统计结果。分别统计已经检测的文件数量、未检测的文件数量以及检测结果当中各个类别的数量。这里的数量展示是与检测同步进行的，即根据检测状况动态更新数据。

（2） 图表统计

图表统计是统计功能的延伸，为了更加直观的显示统计功能数量的增长及其相互之间的对比情况。这里设计展示了两个图形，分别是常见的柱形图以及饼图，柱形图展示检测文件总数，以及各个类别的数量，根据检测过程动态更新数据并展示。饼图展示各个类别的数量，方便观察各个类别之间的比例情况，与柱形图一样，都是动态同步实时更新数据与图形的。

（3） 具体检测信息

具体检测信息用于观察当前上传的所有文件的上传进度、检测进度、检测结果等。同时，精准捕捉某个文件的上传状态，可以重新上传与停止上传，防止因为网络等意外情况出现的上传失败。当前检测的结果记录会被存储到数据库中，并在历史模块中展示。

（4） 历史检测信息

历史检测信息用于展示过往所有文件的检测记录，主要包括检测结果、检测时间等。可以通过条件查询指定的历史记录，并将其展示在历史检测信息。

(5) 导出模块

在历史检测信息中可将当前列表中展示的历史记录以 Excel 表格的形式导出下载到本地进行分析处理。

3.4 数据库设计

在设计完系统的整体架构和功能模块后，系统所需的数据库也需要进行设计。本系统主要用数据库的表来保存和展示漏洞检测结果，具体分为当前检测的文件结果表 3.1 和历史检测的文件结果表 3.2。

表 3.1 当前检测结果表 code_current

列名	数据类型	长度	非空	主键	注释
id	varchar	30	是	是	主键 id
file_name	varchar	30	是	否	文件名称
status	varchar	10	是	否	检测状态
result	varchar	10000	是	否	检测结果

当前检测结果表主要用来存储当前检测文件的数据，包含文件 id 字段、文件名称字段、检测状态字段和检测结果字段。其中 status 字段为当前检测文件的状态，有“完成”和“未完成”两个值。

表 3.2 历史检测结果表 code_history

列名	数据类型	长度	非空	主键	注释
id	varchar	30	是	是	主键 id
file_name	varchar	30	是	否	文件名称
create_time	varchar	30	是	否	检测时间
result	varchar	10000	是	否	检测结果

历史检测结果表主要用来存储历史检测文件的数据，包括文件 id 字段、文件名称字段、检测时间字段和检测结果字段，该表供系统使用者查询和下载历史检测结果数据。

3.5 本章小结

本章节对基于深度学习的源代码漏洞检测系统进行了总体设计。首先，对系统的需求分析进行阐述，设计了系统的总体架构，介绍了前后端框架与请求/响应设计、前端功能设计和后端功能设计，并对前后端的流程进行了详细介绍；然后，对系统功能模块进行了设计，包括文件上传模块、源代码预处理模块、漏洞检测模块和结果显示模块；最后，对系统所用的数据库表进行了设计，包括当前检测结果表和历史检测结果表。其中，融入 DistilBERT-LSTM

和多项朴素贝叶斯的漏洞检测方法、多特征融合的漏洞检测方法是本文的重点模块，将在第六章中给出详细具体的方案。

第四章 基于 DistilBert-LSTM 与多项朴素贝叶斯的源代码漏洞检测方法

第三章给出了基于深度学习的源代码漏洞检测系统的总体设计。结合总体设计，本章提出了一种基于 DistilBert-LSTM 和多项朴素贝叶斯的漏洞检测方法 DBM-MNB，该方法适用于对源代码的深度表征情形，下面对方法的详细设计进行具体介绍。

4.1 问题分析

根据第二章介绍的源代码漏洞检测方法的内容可知，漏洞检测已经成为软件安全性的重要保障，漏洞检测方法越来越受到软件开发者和使用者的关注。通过对现有的漏洞检测方法的研究可以发现，现有的漏洞检测方法通常是提取代码的一类特征，然后用单个学习模型进行训练和测试，忽略了组合学习模型对漏洞检测带来的优化作用，一方面可以增加漏洞信息表征的深度，另一方面可以提高代码信息表征的完整度。如何利用组合模型提升漏洞检测的准确率，成为目前亟待解决的问题。

基于组合学习模型的漏洞检测方法，已经有不少学者进行了研究。文献^[63]提出了一种基于卷积神经网络(Convolutional Neural Network, CNN)和全局平均池化(Global Average Pooling, GAP)可解释性模型的源代码漏洞检测方法。在源代码预处理中对部分自定义标识符进行归一化，构建 CNN-GAP 神经网络模型，识别出包含 CWE-119 缓冲区溢出类型漏洞的函数。杨宏宇等人^[64]提出了基于结构化文本及代码度量的漏洞检测方法，将自注意力机制的神经网络、深度神经网络并行对漏洞数据集进行检测，再用支持向量机优化检测结果，对源代码漏洞的存在性检测性能得到了明显的提升。Huang 等人^[65]提出一种新的自动漏洞分类模型(TFI-DNN)，该模型建立在词频-逆文档频率(TF-IDF)^[66]、信息增益(Information Gain, IG)^[67]和深度神经网络(Deep Neural Network, DNN)^[68]之上，实验证明模型对美国国家漏洞数据库的检测具有有效性。上述工作通过组合深度学习模型进行漏洞检测，达到了一定检测效果，但模型对源代码的信息挖掘能力有待提高。

为了提高源代码漏洞信息表征的完整度、降低漏洞检测的漏报率和误报率，本章提出一种基于 DistilBert-LSTM 和多项朴素贝叶斯(DistilBert-LSTM Multinomial Naive Bayes, DBM-MNB)的漏洞检测模型，主要工作如下：

- 1)本章提出 DBM 模型实现漏洞函数的源代码文本深度表征，捕捉漏洞的局部关键特征。

- 2) DBM 模型可以捕获漏洞的全局时间特征, 深度挖掘漏洞语句间的依赖关系。
- 3) 针对漏洞检测过程中的难样本, 本方法采用 MNB 对模型检测结果进行优化, 从而提高漏洞检测准确率, 降低漏洞检测的误报率和漏报率。
- 4) 实验结果表明, 本章提出的 DBM-MNB 模型可以有效检测漏洞是否存在, 提升漏洞检测的性能指标。

4.2 DBM-MNB 模型框架

为了实现漏洞的检测, 本章提出一种 DBM-MNB 模型对处理后的函数源代码进行检测。该模型主要由数据预处理、DBM 模型和 MNB 模型三个部分组成。模型架构如图 4.1 所示。

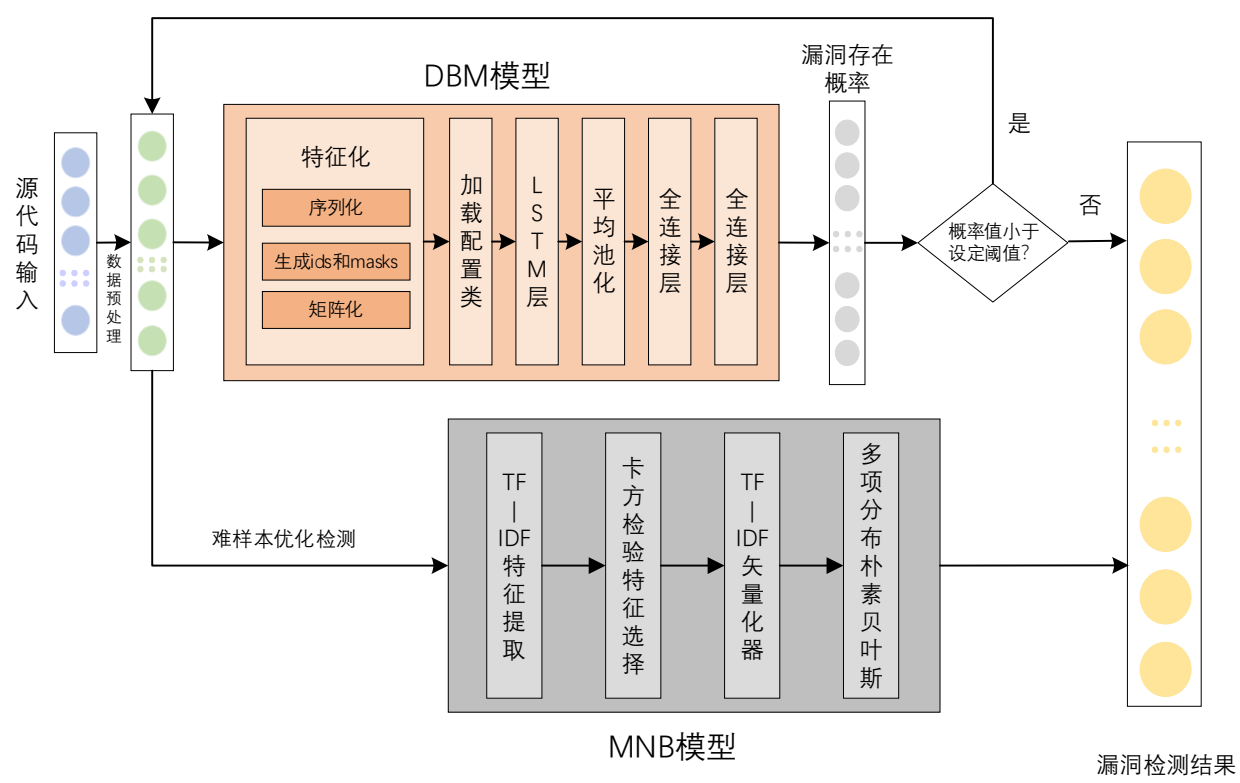


图 4.1 基于 DistilBert-LSTM 与多项朴素贝叶斯的源代码漏洞检测模型框架图

4.2.1 数据预处理

函数中包含许多无用信息, 会影响模型检测的结果, 因此需要对函数源代码进行相关预处理操作。首先, 对函数源代码进行清洗, 删除源代码中的空行和注释等, 防止这些无用信息被表征, 增加模型的输入维度; 其次, 将清洗后的代码保存在列表中; 然后, 将有漏洞的函数设置标签为“1”, 无漏洞的函数设置标签为“0”; 最后, 按照 7: 3 比例划分训练数据集和测试数据集。

4.2.2 DBM 模型

将经过数据预处理的源代码数据输入到 DBM 模型中进行特征化处理, 利用 DistilBert 模型捕获漏洞特征之间的依赖性, 添加 LSTM 层捕捉全局时间特征, 使用平均池化将网络输出汇总为一个向量。最后使用两个全连接层对局部特征进行综合处理, 得到漏洞检测结果, 即函数是否存在漏洞。

为了更好地捕获函数漏洞语句之间的依赖性, 使用 DBM 模型进行预训练。这个过程分为特征矩阵的构建和模型的构建。

一、构建函数的特征矩阵

对清洗后的数据, 包括训练数据集和验证数据集, 构建可以直接输入模型的特征矩阵 M :

(1) 选择序列的最大长度 L 。基于函数的复杂性, 为了增加向量化的完整度, 选择最大序列长度为 300。

(2) 序列化。对源代码进行序列化, 在这过程中 DistilBert 将未知单词拆分为子标记, 直到找到已知的一元组, 然后使用序列化模型插入特殊标记;

(3) 生成函数源代码的 ids 和 masks。根据序列化后的文本生成对应的单词序列码 ids 和掩码 masks, 对于较简单的函数, ids 和 masks 的长度不够时将用“0”进行填充;

(4) 矩阵化。将生成的 ids 和 masks 形成一个张量, 以获得形状为 $[2(ids, masks) \times n \times L]$ 的特征矩阵 M , 其中 n 表示数据中的函数数量, L 表示序列长度。

二、构建 DBM 模型

为了高效捕获函数漏洞的关键特征、提升漏洞检测准确率, 本文构建了一种 DBM 模型, 模型的架构如图 4.2 所示:

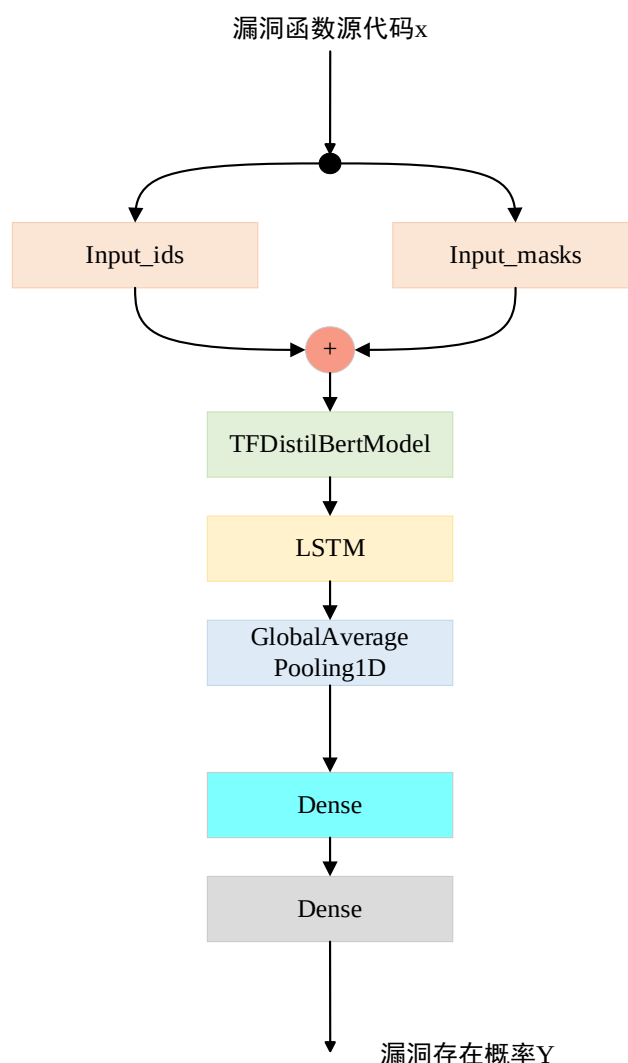


图 4.2 DBM 模型架构图

(1) 模型的输入层是两层，分别是 input_ids 和 input_masks，输出层数是 300。

(2) 加载 TFDistilBertModel 配置类，用于根据指定的参数实例化 DistilBert 模型，设置 dropout 为 0.1，attention_dropout 为 0.1。

(3) 为了捕获漏洞的全局时间特征，在模型上加一层 LSTM 层，有效提升模型的全局时间记忆能力。

(4) 使用平均池化 GlobalAveragePooling1D 将上述神经网络的输出汇总到一个向量中。

(5) 添加两个最终的 Dense 层来预测每个函数漏洞存在的概率，第一个 Dense 层采用激活函数 ReLU 来放大特征间的差异，第二个 Dense 层采用 softmax 进行归一化处理，实现漏洞存在性检验，输出漏洞存在的概率，计算公式如下：

$$\begin{aligned}
 Y &= \text{soft max}(MW^T) \\
 &= \frac{e^{MW^T}}{e^{MW^T} \cdot E_1}
 \end{aligned} \tag{4.1}$$

其中， M 表示特征矩阵， W 表示权重矩阵， E_1 是用于合并计算的单位矩阵。

基于 DBM 的漏洞检测模型实现方法如算法 1 所示。

算法 1 基于 DBM 的漏洞检测模型实现方法

输入：训练集 X_{train} ，测试集 X_{test} ，标签集 Y_D

输出：漏洞存在性检验结果 $\hat{Y}_D$

步骤 1：数据预处理

1 对训练集 X_{train} 及测试集 X_{test} 进行数据清洗，获得 X_{train} 及 X_{test}

2 对数据集进行序列化，生成 X_{train_ids} 、 X_{train_masks} 、 X_{test_ids} 及 X_{test_masks}

步骤 2：构建模型

3 加载 TFDistilBertModel 配置类，实例化 DistilBert 模型

4 添加 LSTM 层、平均池化层和全连接层，采用 softmax 函数作为分类器

步骤 3：训练模型

5 while 未达到预设训练轮数 do

6 while 未达到迭代次数 do

7 设置单次训练样本数 batch_size，以小批次数据集 batch 作为模型输入

8 计算交叉熵损失函数

9 使用 Adam 优化器更新模型参数

10 end while

11 设置验证集，使用验证集验证模型并进行参数微调

12 end while

步骤 4：保存模型

13 保存参数微调后的模型

步骤 5：验证模型

14 载入已保存模型，使用测试集测试该模型

15 return 测试集中源代码漏洞检测结果

4.2.3 MNB 模型

DBM 模型中最重要的部分是加载 TFDistilBertModel 配置类实例化 DistilBert 模型，且 DistilBert 模型关键模块为 Transformer，其核心为自注意力机制，该机制对函数源代码进行信息表征时重点在于捕捉漏洞特征内部的相关性，在漏洞特征差异不明显时，该模型的效果将受到限制。

由于函数漏洞存在检测本质上是一个二分类问题，所以在 DBM 模型输出结果之后，作为补充，使用 MNB 算法进一步地检测会提升检测效果。本文选择基于 MNB 的检测优化算法，原因如下：（1）DBM 模型效果良好需要庞大的数据集进行训练，本文整理的漏洞数据集数量对于该模型的训练量来说相对不足，但朴素贝叶斯分类器模型简单，对数据集数量要求不高；（2）DBM 在捕获函数语句特征信息间的相关性过程中，在特征不明显时，无法区分漏

南京邮电大学硕士研究生学位论文第四章 基于 DistilBert-LSTM 与多项朴素贝叶斯的源代码漏洞检测方法

洞特征和非漏洞特征，而朴素贝叶斯分类器将捕获到的每对特征假设为相互独立，再计算后验概率并进行分类，能够有效降低漏洞特征和非漏洞特征差异不明显时 DBM 模型的误判率。由本文实验结果可知，朴素贝叶斯分类器可提升漏洞存在检测效果。

本方法的 MNB 模型构建如下：

(1) 特征工程。对数据预处理之后的数据，使用限制为 10,000 个单词的 TF-IDF 矢量化器捕获一元词组 **unigrams** 和二元词组 **bigrams**，执行卡方检验以确定特征和目标是否独立，仅保留卡方检验中具有特定 p 值的特征。

(2) 多项朴素贝叶斯分类器。使用多项朴素贝叶斯分类器独立考虑每个函数特征，计算每个漏洞存在的概率，然后预测源代码是否存在漏洞。

(3) 获取漏洞检测结果。在特征矩阵上训练分类器，然后在预处理后的测试集上对其进行测试。为此，构建机器学习 **pipeline**，将转换列表和最终估计器顺序应用。将 TF-IDF 矢量化器和朴素贝叶斯分类器放入 **pipeline** 中，预测漏洞存在概率。

MNB 模型构建方法如算法 2 所示。

算法 2 MNB 模型构建方法
输入：训练集 X_{train} ，测试集 X_{test} ，标签集 $\hat{Y}_D$
输出：漏洞存在性检验结果
步骤 1：数据预处理
1 对训练集 X_{train} 及测试集 X_{test} 进行数据清洗，获得 X_{train} 及 X_{test}
步骤 2：分类模型构建
2 初始化 TF-IDF 向量化器，设置最大特征数为 10,000，词组数为 1 和 2，记为 Vectorizer
3 设置卡方检验的阈值 p 为 0.95
4 对捕获的每个特征执行卡方检验进行特征选择
5 初始化多项朴素贝叶斯分类器，记为 Classifier
6 将 Vectorizer 和 Classifier 合并成 Pipeline ，得到分类模型
步骤 3：训练模型
7 while 未达到预设训练轮数 do
8 while 未达到迭代次数 do
9 设置单次训练样本数 batch_size ，以小批次数据集 batch 作为模型输入
10 计算交叉熵损失函数
11 计算后验概率，即漏洞存在概率，记为 P
12 end while
13 设置验证集，使用验证集验证模型并进行参数微调
14 end while
步骤 4：保存模型
15 保存参数微调后的模型
步骤 5：验证模型
16 载入已保存模型，使用测试集测试该模型

4.3 实验与分析

4.3.1 实验数据集

本文的实验数据集来自 CVE (Common Vulnerabilities & Exposures, 简称 CVE) 漏洞数据^[73], 涵盖了 2002 年到 2019 年随机抽取的栈溢出、SQL 注入等各类漏洞数据, 将漏洞整理成 json 格式, 每条数据具有的属性字段如表 4.1 所示。

表 4.1 数据集属性字段表

特征	在 json 文件中的字段名	属性描述
漏洞复杂性	access_complexity	反映利用软件函数滥用漏洞所需的攻击复杂性
身份验证必要性	authentication_required	是否需要身份验证才能利用该漏洞
提交 ID	commit_message	代码仓库中的提交 ID, 表示小版本
CVE ID	cve_id	常见漏洞 ID
编程语言	lang	项目编程语言
漏洞分类	vulnerability_classification	漏洞类型
修复前函数	func_before	漏洞修复前的函数 (如果“vul”标记为“1”, 则为漏洞函数)
修复后函数	func_after	漏洞修复后的函数
值	vul	“1”表示易受攻击的函数, “0”表示非易受攻击的函数

对于上述特征, 实验使用 vul 和 func_before 两个字段, 当 vul 为 1 时, func_before 的值就是有漏洞的函数, 当 vul 为 0 时, func_before 的值就是没有漏洞的函数, 共筛选出 26097 条数据进行检测。其中, vul 为 1 和 vul 为 0 的数量比例是 11: 15, 训练数据集为 18276 条, 测试数据集为 7821 条。

4.3.2 实验设置

本章实验在如下实验环境下进行。硬件环境: Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz 2.59 GHz, GPU 为 NVIDIA GeForce GTX 1650, 运行内存为 16.0GB; 软件环境: 操作系统为 64 位 Window 10, 集成开发环境 PyCharm 2021.1.2 x64, 编程语言为 Python 3.6.2, 深度学习框架为 Tensorflow 1.14.0+Keras2.2.5。

经数据预处理后, 先通过 DBM 模型进行检测, 该模型参数设置如表 4.2 所示。

表 4.2 DBM 模型参数表

层名称	输出维度	参数设置	上层操作名称
input_ids (InputLayer)	[(None, 300)]	0	[]
input_masks (InputLayer)	[(None, 300)]	0	[]
tf_distil_bert_model (TFDistilBertModel)	TFBaseModelOutput(last_hidden_state=(None, 300, 768), hidden_states=None, attentions=None)	663628 80	['input_ids[0][0]', 'input_masks[0][0]']
lstm(LSTM)	(None, 300, 768)	472466 4	['tf_distil_bert_model[0][0]']
global_average_pooling1d (GlobalAveragePooling1D)	(None, 768)	0	['lstm[0][0]']
dense (Dense)	(None, 64)	49216	['global_average_pooling1d[0][0]']
dense_1 (Dense)	(None, 2)	130	['dense[0][0]']

4.3.3 评价指标

为了评估本实验模型的性能, 将采用如下指标进行评估:

(1) 准确率 (Accuracy, 简称 ACC): 准确率表示预测正确的样本占总样本的比率, 其值越高, 则函数漏洞存在性检测性能越好。

$$ACC = \frac{TP + TN}{TP + TN + FP + FN} \quad (4.2)$$

(2) 精确率 (Precision, 简称 P): 精确率表示被预测为函数漏洞的样本中预测正确的样本比率, 其值越高, 则函数漏洞存在性检测效果越好。

$$P = \frac{TP}{TP + FP} \quad (4.3)$$

(3) 召回率 (True Positive Rate, 简称 TPR): 实际检索到的漏洞样本总数的比例, 其值越高, 则函数漏洞存在性检测对异常函数漏报率越低。

$$TPR = \frac{TP}{TP + FN} \quad (4.4)$$

(4) 误报率 (False Positive Rate, 简称 FPR): 实际为正常的样本中被预测为漏洞的样本比率, 其值越低, 则函数漏洞存在性检测对正常样本判别效果更好。

$$FPR = \frac{FP}{TN + FP} \quad (4.5)$$

(6) F1 分数 (F1-Score, 简称 F1): 精确率和召回率的调和平均值, 反映模型整体表现情况, 其值越高, 则漏洞存在性检测综合判别性能越好。

$$F1 = \frac{2 \times P \times TPR}{P + TPR} \quad (4.6)$$

其中, TP 表示被预测为有漏洞的函数漏洞样本数量, TN 表示被预测为无漏洞的无漏洞函数样本数量, FP 表示被预测为有漏洞的无漏洞函数样本数量, FN 表示被预测为无漏洞的有漏洞函数样本数目。

4.3.4 对比实验与结果分析

(1) DBM-MNB 与子模型对比

首先, 为了验证本章方法可有效检测漏洞的存在性, 将本章提出的 DBM-MNB 模型同 DistilBert+MNB 模型和 DistilBert 模型的性能评估结果进行对比。实验模型如下:

DistilBert 模型: 加载 Transformer 的配置类, 将数据直接输入模型进行检测, 输出结果。

DistilBert+MNB 模型: 将数据先通过 DistilBert 模型进行检测, 对于检测结果较差的数据再输入到多项朴素贝叶斯分类器中进行检测, 输出最终的结果。

表 4.3 DBM-MNB 模型和子模型的性能评估结果对比

模型	ACC	P	TPR	FPR	F1	AUC
DistilBert	79.34%	74.12%	75.26%	19.48%	81.33%	79.78%
DistilBert + MNB	82.63%	83.38%	82.74%	15.25%	83.41%	85.24%
DBM-MNB	86.77%	86.33%	88.87%	14.74%	86.56%	93.20%

表 4.3 展示了上述模型性能评估结果对比。横向对比, 相同模型的深度, DistilBert+MNB 模型相比于 DistilBert 模型能有效提升漏洞检测的准确率, 说明添加 MNB 模型能够有效提升漏洞检测的性能。纵向对比, DBM-MNB 模型相较于 DistilBert 模型和 DistilBert+MNB 模型性能更优, 说明增加模型深度且使用 LSTM 层可以更好地捕获全局时间特征, 同时有效降低漏洞检测的漏报率和误报率, 获得更高的准确率、精确率和 F1 分数, 以及更优的 ROC 曲线的 AUC 值。

(2) DBM-MNB 与基于 NLP 的模型对比

Model 1: 构建 TF-IDF 矢量化器进行特征工程, 执行卡方检验进行特征选择, 使用多项朴素贝叶斯分类器, 最后将 TF-IDF 矢量化器和分类器合并到一个 sklearn 管道对函数源代码进行漏洞存在性检测。

Model 2: 单层 BiLSTM, 构建词嵌入向量进行特征工程, 使用单层双向 LSTM 有效提取函数源代码的时序信息, 对序列中的单词顺序进行建模。

Model 3: 双层 BiLSTM, 构建词嵌入向量进行特征工程, 使用双层双向 LSTM 有效提取函数源代码的时序信息, 在对序列中的单词顺序进行建模。

Model 4^[55]: BiLSTM+注意力机制, 构建词嵌入向量进行特征工程, 使用双层 LSTM 有效提取函数源代码的时序信息, 采用注意力机制增加漏洞代码的细粒度分析。

Model 5^[74]: Bi-GRU, 构建词嵌入向量进行特征工程, 使用 Bi-GRU 有效提取函数源代码序列的长期依赖关系, 处理双向的顺序信息, 挖掘代码中的结构信息。

表 4.4 DBM-MNB 模型和其他模型的性能评估结果对比

模型	ACC	P	TPR	FPR	F1	AUC
Model 1	78.78%	79.37%	62.70%	11.43%	77.14%	84.17%
Model 2	74.58%	73.90%	67.44%	20.29%	73.72%	72.41%
Model 3	72.42%	71.90%	69.36%	23.37%	72.00%	81.91%
Model 4	74.34%	73.67%	73.71%	25.69%	73.79%	73.96%
Model 5	63.77%	71.09%	83.61%	36.79%	68.22%	70.43%
DBM-MNB	86.77%	86.33%	88.87%	14.74%	86.56%	93.20%

表 4.4 给出了不同学习模型漏洞检测性能评估的结果对比。虽然机器学习模型的构建相对简单, 但是在本数据集上的实验效果优于单层 BiLSTM、双层 BiLSTM、BiLSTM+注意力机制和 Bi-GRU, 可见模型越复杂, 漏洞检测效果不一定越好; 单层 BiLSTM 相较于双层 BiLSTM 和 BiLSTM+注意力机制在本数据集上的实验效果相差不大, 说明随着模型深度的增加, 漏洞检测效率不一定能够相应地提高; DBM-MNB 较其余模型, 准确率高达 86.77%, 精确率提升至 86.33%, 误报率降至 14.74%, F1 分数增至 86.56%, 同时 DBM-MNB 模型 AUC 值更高, 整体呈现更佳的性能。

(3) DBM-MNB 与 4.1 节模型对比

为了体现 DBM-MNB 模型解决了 4.1 节中文献存在的问题, 将其与文献[63]、文献[64]和文献[65]中的模型进行对比。与各文献模型的对比, 均使用原文采用的数据集, 即与文献[63]的对比采用 CWE-119 数据集, 与文献[64]的对比采用 CWE-134 数据集, 与文献[65]的对比采用 NVD 2000-2016 数据集, 具体实验结果如表 4.5、表 4.6 和表 4.7 所示。

表 4.5 DBM-MNB 模型和文献[63]的性能评估结果对比

模型	ACC	P	TPR	FPR	F1
文献[63]	84.90%	92.35%	76.10%	6.30%	83.44%
DBM-MNB	90.32%	91.53%	78.50%	4.58%	86.56%

表 4.6 DBM-MNB 模型和文献[64]的性能评估结果对比

模型	ACC	P	FPR	F1
文献[64]	84.22%	67.41%	9.92%	66.46%
DBM-MNB	83.22%	84.49%	3.13%	75.98%

表 4.7 DBM-MNB 模型和文献[65]的性能评估结果对比

模型	ACC	P	F1	TPR
文献[65]	0.87	0.85	0.63	0.82
DBM-MNB	0.89	0.90	0.81	0.76

由上述三个表格可知本文提出的一种基于 DistilBert-LSTM 与多项朴素贝叶斯的源代码漏洞检测方法 DBM-MNB 具有较优的性能，可以解决 4.1 节中源代码漏洞信息挖掘深度不足的问题，并能获得较高的漏洞检测准确率和较低的误报率、漏报率。因此该漏洞检测方法可以满足第三章中漏洞检测模块的设计需求。

4.4 本章小结

本章首先对相关漏洞检测方法的问题进行了研究和分析，在尽可能保留漏洞特征信息的前提下，为了提升漏洞检测准确率，降低漏洞检测漏报率、误报率，提出了一种基于 DistilBert-LSTM 与多项朴素贝叶斯的漏洞检测方法 DBM-MNB。首先，在传统 DistilBert 模型上增加 LSTM 层用于捕获漏洞的全局时间特征；然后，通过模型检测得出漏洞的存在概率；最后，对难样本采用多项朴素贝叶斯分类器进一步学习分类，得出最后的漏洞检测结果。实验结果表明，LSTM 层和多项朴素贝叶斯分类器能有效提升漏洞检测的准确率，体现更好的模型性能。大量的对比实验表明，本章提出的 DBM-MNB 模型相较于其他学习模型能呈现更优的检测性能指标，有效降低漏洞检测的漏报率和误报率。

第五章 基于多特征融合的源代码漏洞检测方法

结合第三章的系统架构，本章提出了一种基于多特征融合的漏洞检测方法，适用于对源代码的全面表征情形。该方法可以根据需求，与第四章的漏洞检测方法进行切换使用，下面对方法的设计过程进行详细阐述。

5.1 问题分析

根据第二章介绍的源代码漏洞检测方法的内容可知，源代码的深度学习表征方式会直接影响到漏洞检测的效果。这种表征就是将源代码表示为不同的形式，如抽象语法树、图以及文本等，从中提取出函数的结构关系和依赖关系等，从而挖掘出漏洞的关键特征。通过对现有进展的研究可以发现，漏洞检测基于文本信息可以带来更好的效果，而图等特征会增加不必要的分支解析而降低漏洞检测的准确率。因此，如何利用源代码文本信息提升漏洞检测的效率，成为目前亟待解决的问题。

Wang 等人^[69]发现仅使用词法分析得到的漏洞特征^[70]具有较低的性能，将卷积神经网络（Convolutional Neural Network, CNN）与自然语言处理（Natural Language Processing, NLP）相结合得出源代码数据集的特征，提出了一个系统化的特征提取框架 PreNNsem，用于漏洞检测。文献[71]提出了一种基于深度学习的漏洞检测工具 Vudenc，该工具应用 Word2vec 模型来识别语义相似的代码标记并提供向量表示。然后使用 LSTM 网络在不同的细粒度上对可能含有漏洞代码的 token 序列进行分类，分析出易受攻击代码的特定区域，并为其提供置信度预测。文献[72]从源代码文本中提取特征，使用机器学习算法来预测输入的漏洞。该方法可以提取程序中与漏洞流相关的所有特征，并去除与漏洞流无关的特征，这个特征提取方法具有较高的可重用性和更好的性能。以上研究工作都是基于文本信息对源代码进行漏洞检测，但通常是基于一种文本特征，对代码的表征能力不足，没有完全捕获漏洞的关键特征。

为了全面表征源代码的漏洞特征，本章提出一种基于多特征融合的源代码漏洞检测方法（CHR-CodeBERT）。该方法提取源代码的 CNN 特征、HAN 特征和 RNN 特征，在不同细粒度上挖掘漏洞信息，通过加性注意力机制将三个特征进行融合，再用 CodeBERT 模型进行分类，检测出源代码是否存在漏洞。

5.2 模型框架

为了实现漏洞的高效检测，本章提出一种 CHR-CodeBERT 模型对处理后的函数源代码进行检测，该模型主要由数据预处理、CNN 特征提取、HAN 特征提取、RNN 特征提取、基于加性注意力机制的特征融合以及 CodeBERT 分类模型等内容组成。具体流程将在接下来的小节中进行介绍。

5.2.1 模型总体框架

CHR-CodeBERT 的总体框架如图 5.1 所示，对每条源代码先进行数据预处理操作，然后提取 CNN、HAN 和 RNN 特征，根据设计的加性注意力机制模块进行特征融合，得到一个特征矩阵。同时，对预处理后的数据进行添加特殊标记、生成掩码 `masks`、`padding` 处理、生成 `ids`、生成 `segments` 等操作，从而生成一个特征矩阵。将以上操作获得的两个特征矩阵作为 CodeBERT 的模型输入，添加平均池化层和两个全连接层实现对源代码中漏洞的检测，具体内容将在下面的小节中介绍。

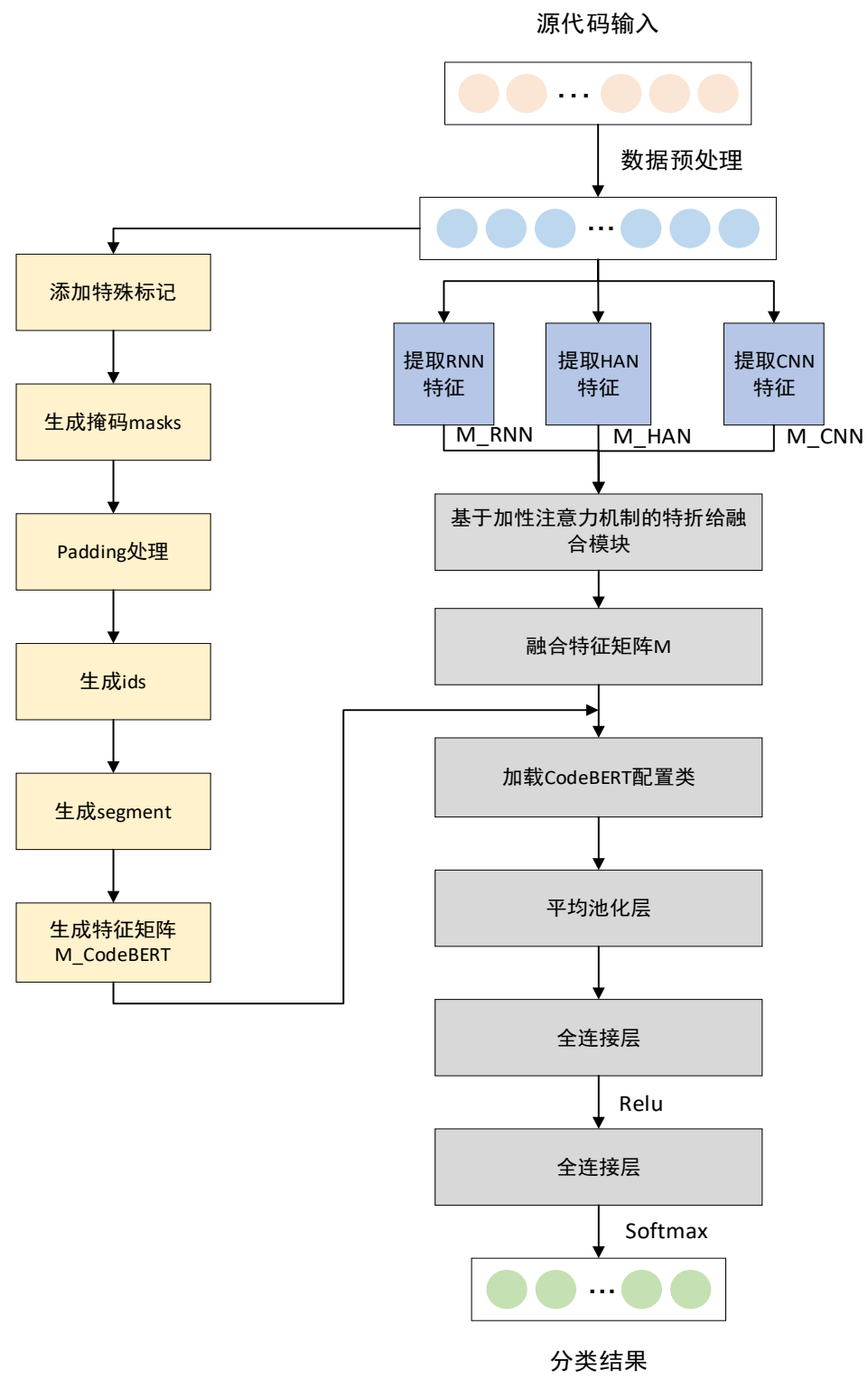


图 5.1 CHR-CodeBERT 模型框架图

5.2.2 数据预处理

由于代码中有许多冗余信息会影响到模型检测效果，所以需要先对其进行预处理操作，对所有源代码数据进行以下操作：

- (1) 删除源代码中的空行和注释行，防止这些无用信息被表征而增加特征维度。
- (2) 下载 `nlTK_data` 的停用词包和分词包对源代码删除停用词，并用分词包对源代码进行分词操作。
- (3) 将清洗后的源代码用新的列表保存，提取标签数量和源代码数量。
- (4) 对有漏洞的代码设置标签为“1”，没有漏洞的代码设置标签为“0”，训练集和测试集的数据量划分比例为 8：2。

5.2.3 CNN 特征提取模块

本模块提取 CNN 特征，从单词的细粒度上进行提取，设置最大的句子长度为 L 。由于表示漏洞函数的单词通常具有相关性，相比单词同时出现的概率，单词同时出现的概率的比率能够更好地区分漏洞特征。因此，词向量选择 Glove，词嵌入向量维度为 Dim_{word} ，词向量数目为 Num_{word} 。对源代码提取特征矩阵，设置词数量为 P ，每个单词创建的维度为 Q ，则提取

的嵌入矩阵 $M_{CNN} = \begin{bmatrix} x_{00} & x_{01} & x_{02} & \dots & x_{0q} \\ x_{10} & x_{11} & x_{12} & \dots & x_{1q} \\ \dots & \dots & \dots & \dots & \dots \\ x_{p0} & x_{p1} & x_{p2} & \dots & x_{pq} \end{bmatrix}$ 。具体实现方法如算法 5.1 所示。

算法 5.1 CNN 特征提取方法

输入：

经数据预处理后的源代码集 X 和标签集 Y

最大的句子长度为 L

词嵌入向量维度为 Dim_{word}

词向量数目为 Num_{word}

输出： CNN 特征矩阵 M_{CNN}

1 根据 Num_{word} 对源代码进行序列化，得到序列 $Sequences$

2 根据 $Sequences$ 提取单词序号，得到唯一性单词索引 $word_index$ ，长度为 len

3 根据 L 计算得到源代码集向量 $\bar{X}$ 和标签集向量 $\bar{Y}$

4 加载 Glove 6B 100d 词向量对 $\bar{X}$ 进行词嵌入，得到词向量 $word_vector$

5 构建词嵌入矩阵 $M_{embedding} = [len + 1, Dim_{word}]$

6 for (word,index) in $word_index.items()$:

7 $embedding_vector = word_vector.get(word)$

8 If $embedding_vector$ is not None:

9 $\bar{M}_{embedding}[i] = embedding_vector$ //单词不在索引中将设置默认值为 0

10 根据 $word_index, Dim_{word}, M_{embedding}, L, word_index$ 进行嵌入得到特征矩阵

5.2.4 HAN 特征提取模块

上一节的特征提取是在单词级别上的，但源代码的长度不确定，对于篇幅较长的函数 CNN 特征会影响漏洞检测的精度和速度。HAN 使用层次注意力机制从句子（短句/长句）细粒度上对源代码进行特征提取，对句子和句子中的词语使用 BiLSTM+Attention 的方式进行编码得到词向量，主要包括以下部分：

Word Encoder: 先对词汇进行编码，建立词向量。接着用 BiLSTM 从单词的两个方向汇总信息来获取单词的语义表示，因此将上下文信息合并到句子向量中。

Word Attention: 对每句话的词语进行 Attention 操作，最后每句话都有一个特征向量，可以看做句子向量。

Sentence Encoder: 与 Word Encoder 相似，对句子级别使用 BiLSTM 获取上下句的信息。

Sentence Attention: 与 Word Attention 相似，对所有句子进行 Attention 操作，获得每个句子加权平均作为整个特征向量。

设输入为 x ，每个单词创建的维度为 m ，句子中单词数为 $snum$ ，句子长度为 $slen$ ，BiLSTM 隐藏状态数量为 n ，层数为 l ，那么词向量输出为 $woutput = [x * snum, slen, 2 * n]$ 。

词注意力机制向量的权重计算为 $weight_w = [x * snum, slen, l]$ ，根据运行类型为 CPU/GPU 更新权重为 $weights_w$ ，然后对 x 进行 padding 化处理并进行归一化处理，得到此时的词向量权重为

$$weight'_w = \frac{weights_w}{sum(weight_w, m) + 1e - 4} \quad (5.1)$$

其中 $1e - 4$ 是为了优化训练添加的极小值， $sum()$ 表示求和。

对于句子，其向量计算为 $svector = sum(weights_w * woutput, l)$ ，其中 $sum()$ 表示求和。

$soutput = self.BiLSTM(vector)$ ，注意力机制向量的权重计算为 $weight_s = [x, snum, l]$ ，同样根据运行类型为 CPU/GPU 更新权重为 $weights_s$ ，然后对 x 进行 padding 化处理并进行归一化处理，得到此时的句子向量权重为

$$weights_s = \frac{weights_s}{sum(weights_s, n) + 1e-4} \quad (5.2)$$

其中 $1e-4$ 是为了优化训练添加的极小值， $sum()$ 表示求和。

根据词向量和句子向量的权重，计算得到 HAN 提取的特征向量为 $M_{HAN} = sum(weights_s * soutput, l)$ 。

5.2.5 RNN 特征提取模块

本模块提取 RNN 特征，在文档的细粒度上进行提取，从整体上对漏洞函数进行分析。将清洗好的代码切分成一个个的 token，然后将句子转换成数值的 token，保存每个句子的序列长度。设置序列化后的单词索引为 $word_{index}$ ，长度为 len ，每个单词创建的向量维度为 n ，此时得到的嵌入矩阵为 $M_{RNN} = [len+1, n]$ 。该模块的特征提取方法同算法 5.1。

5.2.6 基于加性注意力机制的特征融合模块

根据以上三小节得到的特征矩阵，本小节将其通过加性注意力机制进行融合。首先将三个特征矩阵进行合并，让 M_{CNN} 、 M_{HAN} 、 M_{RNN} 的列数保持一致，得到一个矩阵 M 。将嵌入矩阵转换成加性注意力机制查询、键和值序列，设每条源代码序列构成的长度为 N ，隐藏状态层数为 b ，那么可得到查询为 $q = [q_0, q_1, \dots, q_{N-1}]$ ，键为 $k = [k_0, k_1, \dots, k_{N-1}]$ ，值为 $v = [v_0, v_1, \dots, v_{N-1}]$ 。相应地，可以计算每个查询向量的注意力权重为

$$w_i = \frac{\exp(q_i / \sqrt{b})}{\sum_{j=1}^N \exp(q_j / \sqrt{b})} \quad (5.3)$$

最终求和得到模型融合有加性注意力向量为

$$w = \sum_{a=1}^N w_a q_a \quad (5.4)$$

5.2.7 CodeBERT 分类模块

CodeBert 对编程语言的敏感度大于许多其他深度学习模型，因此本模块使用该模型进行微调实现漏洞分类检测。由于源代码篇幅的长度不定，所以选择较大的序列长度设为 R 。

CodeBERT 会将未知单词拆分成子标记，直到找到已知元组，句子的开始令牌为 `cls_token`，结束令牌为 `sep_token`，未知的单词令牌为 `unk_token`，不在词汇表中的标记无法转换为 `id` 可使用 `pad_token`，掩码令牌为 `mask_token`，然后进行特征提取和特征矩阵的生成，具体步骤如下：

(1) 为清洗后的源代码添加特殊标记，使其成为各个小的子标记，对于未知的单词会标记为 '`<unk>`'，其中第一个 '`<unk>`' 表示句子的开始，已知单词会按其本身标记。而拼接的单词会将其拆分表示且第一个已知单词会加上 $\dot{G}$ 来以示区分，维度不够 R 的标记将由 '`<unk>`' 填充，最后生成长度为 100 的标记序列。例如函数名 `static uint32_t` 会拆分成 ['`<unk>`', '`static`', '`Ġuint`', '`32`', '`_t`', '`<unk>`', '`<unk>`',]。

(2) 对标记序列生成掩码 `masks`，具有标记的单词掩码都是 1，第一个 '`<unk>`' 的 `mask` 是 1，其他的为 0。

(3) 对标记序列中不在词汇表的单词进行 `padding` 处理，生成新的标记序列。

(4) 对新的标记序列生成 `ids`，使用 CodeBERT 预训练的 `tokenizer` 将序列转换成 `ids`。

(5) 对新的标记序列生成 `segments`，句子结束的地方往后 `seg` 值为 1，前面 `seg` 值为 0。

(6) 将 `masks`、`ids` 和 `segments` 合并到一个向量中，训练集数量为 N_{train} ，得到训练集特征矩阵为 $M_{train} = [masks, ids, segments]_{N_{train} \times R}$ 。

将 5.2.6 节得到的特征矩阵和 M_{train} 作为 CodeBERT 模型的输入，添加 CodeBERT 的设置参数，添加平均池化层 `GlobalAveragePooling1D()`，添加全连接层 `Dense1`，激活函数为 `Relu`，添加全连接层 `Dense2`，激活函数为 `softmax`，对源代码进行漏洞分类，分类结果为当前代码是否具有漏洞。具体实现如算法 5.2 所示。

算法 5.2 基于 CHR-CodeBERT 的源代码漏洞检测

输入：

经过数据预处理后的源代码训练集 $\{X_{train}, Y_{train}\}$

经过数据预处理后的源代码测试集 $\{X_{test}, Y_{test}\}$

多特征融合向量 v

训练轮数 $epochs$

批大小 $batch$

验证集比率 r

学习率 lr

输出：

基于多特征融合的源代码漏洞检测 `CHR-CodeBERT()`

步骤 1：训练 `CHR-CodeBERT()`

```
1      以  $r$  划分训练集  $\{\bar{X}_{train}, \bar{Y}_{train}\}$  与验证集  $\{X_{val}, Y_{val}\}$ 

2      以批大小  $batch$  划分  $\{\bar{X}_{train}, \bar{Y}_{train}\}$  以获得  $train\_data$ 

3      将  $ids$ 、 $masks$ 、 $segments$  和  $v$  添加至输入层
4      for epoch in range( $epochs$ ):
5          for step, ( $batch\_x, batch\_y$ ) in enumerate( $train\_data$ ):
6              生成单元内的  $ids$ 、 $masks$  和  $segments$ , 添加  $v$ 
7              加载 TFRobertaModel 配置类, 实例化 CodeBERT 模型
8              添加 GlobalAveragePooling1D()
9              添加 Dense(), 激活函数为 Relu
10             添加 Dense(), 分类器为 softmax
11             计算交叉熵损失函数
12             采用  $Adam(lr, CHR - CodeBERT)$  优化器更新模型参数

13     通过  $\{X_{val}, Y_{val}\}$  进行验证和参数微调

14     return  $CHR - CodeBERT()$ 
    步骤 2: 保存 CHR-CodeBERT
16     保存参数微调后的模型  $CHR - CodeBERT()$ 
    步骤 3: 测试及评估 CHR-CodeBERT
17     加载  $CHR - CodeBERT()$ 

18     通过  $Y_{test}$  与  $CHR - CodeBERT(X_{test})$  进行模型测试及评估
```

5.3 实验与分析

5.3.1 实验数据集

为了验证本章漏洞分类模型的有效性, 将在以下数据集上进行实验:

(1) 由 Saikat 等人从 Linux Debian Kernel 和 Chromium 的 vulnerability fixed patches 中构造出的源代码, 这个数据集包括两个 json 文件, 分别为有漏洞的源代码和无漏洞的源代码, 编程语言为 C/C++。对于每个 patch, 选取其从漏洞版本到修复版本过程中被修改过的源文件 (.c.cpp) 和头文件 (.h)。对于被修改的 function, 修改前标注为 vulnerables, 修改后标注为 non-vulnerable, 其余没有发生变化的函数都标注为 non-vulnerables。本章进行实验时先对两个 json 文件打标签, 有漏洞函数标签为“1”, 数量为 2240, 无漏洞函数标签为“0”, 数量为 20494。

(2) 从 FFMpeg+Qemu 中构造的数据集, 是一个 json 文件, 里面包括 4 个字段, 具体信息如表 5.1 所示。

表 5.1 FFMpeg+Qemu 数据集字段属性表

project	表示项目来源，值为 FFMpeg 或者 Qemu
commit_id	表示源代码编号，由数字和字母组成
target	表示是否有漏洞，“1”为有漏洞，“0”为无漏洞
func	表示一个函数的代码内容，包括函数定义那一行

本章进行实验时选用 target 和 func 两个字段的数据，该数据集有漏洞函数数量为 12460，无漏洞函数数量为 14858。

5.3.2 实验设置

本文实验在如下实验环境下进行。

表 5.2 实验环境表

CPU	Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz 2.59 GHz
GPU	NVIDIA GeForce GTX 1650
运行内存	16.0GB
操作系统	64 位 Window 10
集成开发环境	PyCharm 2021.1.2 x64
编程语言	Python 3.6.2
深度学习框架	Tensorflow 1.14.0+Keras2.2.5, Torch1.13.1+cu116

5.3.3 评价指标

为了评估本实验模型的性能，将依旧采用准确率 ACC、精确率 P、召回率 TPR、误报率 FPR 和 F1 分数作为指标进行评估。同时，给出各个模型的 ROC 曲线图和 AUC 值，其中 ROC (Receiver Operating Characteristic Curve) 是接收者操作特征曲线，用于评价模型的预测能力，值越高表明模型的预测能力越强。AUC (Area Under Curve) 被定义为 ROC 曲线下的面积，值越高表明该模型的分类效果越好。

5.3.4 对比实验与结果分析

1、CHR-CodeBERT 与基于 NLP 的模型对比

为了验证本章提出的源代码漏洞检测方法的有效性，将基于多特征融合的模型与以下基于 NLP 的模型进行对比：

(1) CNN：对清洗后的代码构建词向量和特征矩阵，使用 CNN 进行分类。具体模型参数如下：

表 5.3 CNN 模型参数设置表

层名称	输出维度	参数设置
input_1 (InputLayer)	[(None, 1000)]	0
embedding (Embedding)	(None, 1000, 100)	6168800
conv1d (Conv1D)	(None, 996, 128)	64128
max_pooling1d (MaxPooling1D)	(None, 199, 128)	0
conv1d_1 (Conv1D)	(None, 195, 128)	82048
max_pooling1d_1 (MaxPooling1D)	(None, 39, 128)	0
conv1d_2 (Conv1D)	(None, 35, 128)	82048
max_pooling1d_2 (MaxPooling1D)	(None, 1, 128)	0
flatten (Flatten)	(None, 128)	0
dense (Dense)	(None, 128)	16512
dense_1 (Dense)	(None, 2)	258

(2) HAN: 对清洗后的代码构建词向量和特征矩阵, 使用 HAN 进行分类。具体模型参数如下:

表 5.4 HAN 模型参数设置表

层名称	输出维度	参数设置
input_2 (InputLayer)	[(None, 15, 100)]	0
time_distributed (TimeDistributed)	(None, 15, 200)	6329600
bidirectional_1 (Bidirectional)	(None, 200)	240800
dense (Dense)	(None, 2)	402

(3) RNN: 对清洗后的代码构建词向量和特征矩阵, 使用 RNN 进行分类。具体模型参数如下:

表 5.5 RNN 模型参数设置表

层名称	输出维度	参数设置
input_1 (InputLayer)	[(None, 1000)]	0
embedding (Embedding)	(None, 1000, 100)	6168800
bidirectional (Bidirectional)	(None, 200)	160800
dense (Dense)	(None, 2)	402

(4) DistilBERT: 对清洗后的代码构建词向量和特征矩阵, 生成对应的掩码 masks 和 ids, 使用 DistilBERT 进行分类, 模型参数同第四章。

(5) VUDENC[71]: 应用 Word2vec 模型来识别语义相似的代码标记并提供矢量表示。然后使用长短期记忆 (LSTM) 网络对检测代码进行分类。

(6) CHR-CodeBERT: 对清洗后的源代码提取多种特征并融合, 输入到 CHR-CodeBERT 模型进行分类。

对于数据集 (1), 实验结果如表 5.6 所示:

表 5.6 CHR-CodeBERT 模型和其他模型在数据集（1）的性能评估结果对比

方法	ACC	P	TPR	FPR	F1	AUC
CNN	52.23%	52.40%	31.82%	35.12%	44.34%	53.35%
HAN	59.34%	68.10%	19.25%	26.94%	60.59%	58.28%
RNN	60.27%	67.42%	12.08%	16.23%	48.62%	50.52%
DistilBERT	66.23%	61.63%	41.19%	25.88%	62.08%	62.65%
CHR-CodeBERT	67.77%	68.13%	63.09%	26.86%	67.77%	68.12%

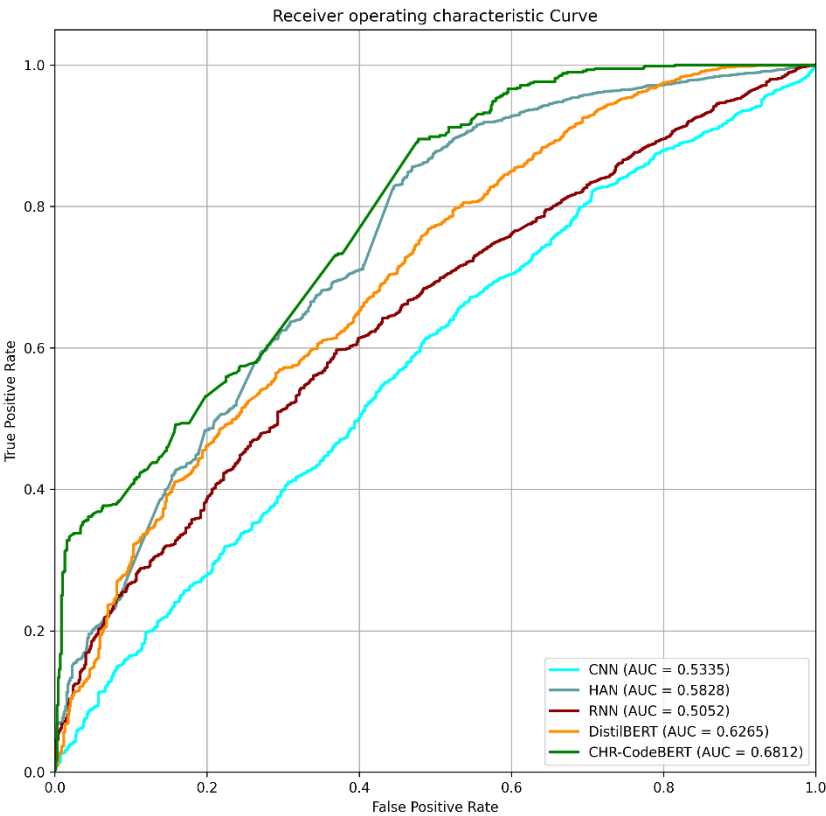


图 5.2 不同学习模型在数据集（1）的 ROC 曲线对比

表 5.6 给出了不同学习模型在数据集（1）进行漏洞检测性能评估的结果对比。在实验数据集（1）的实验效果上，HAN 较 CNN 和 RNN 在总体上呈现更优的指标，可见对漏洞信息的分层挖掘很重要；同时，DistilBERT 较 HAN 模型在整体上具有更优的指标，对于数据集较少的情况下，前者的模型层数更多，对漏洞特征的信息挖掘能力更强；在数据集严重不平衡的情况下，CHR-CodeBERT 与其余模型相比，准确率达到 67.77%，精确率提升至 68.13%，误报率降至 26.86%，F1 分数增至 67.77%。同时，由图 5.2 可知，CHR-CodeBERT 模型 ROC 曲线的 AUC 值更高，整体呈更佳的性能。

对于数据集（2），实验结果如表 5.7：

表 5.7 CHR-CodeBERT 模型和其他模型在数据集（2）的性能评估结果对比

方法	ACC	P	TPR	FPR	F1	AUC
CNN	86.76%	94.%50	52.09%	17.58%	78.98%	75.17%
HAN	78.39%	78.67%	63.33%	11.26%	76.72%	76.03%
RNN	73.97%	73.35%	66.79%	20.76%	73.15%	73.01%
DistilBERT	73.53%	72.88%	65.07%	20.34%	72.56%	72.36%
CHR-CodeBERT	87.30%	86.88%	89.53%	14.30%	87.11%	87.61%

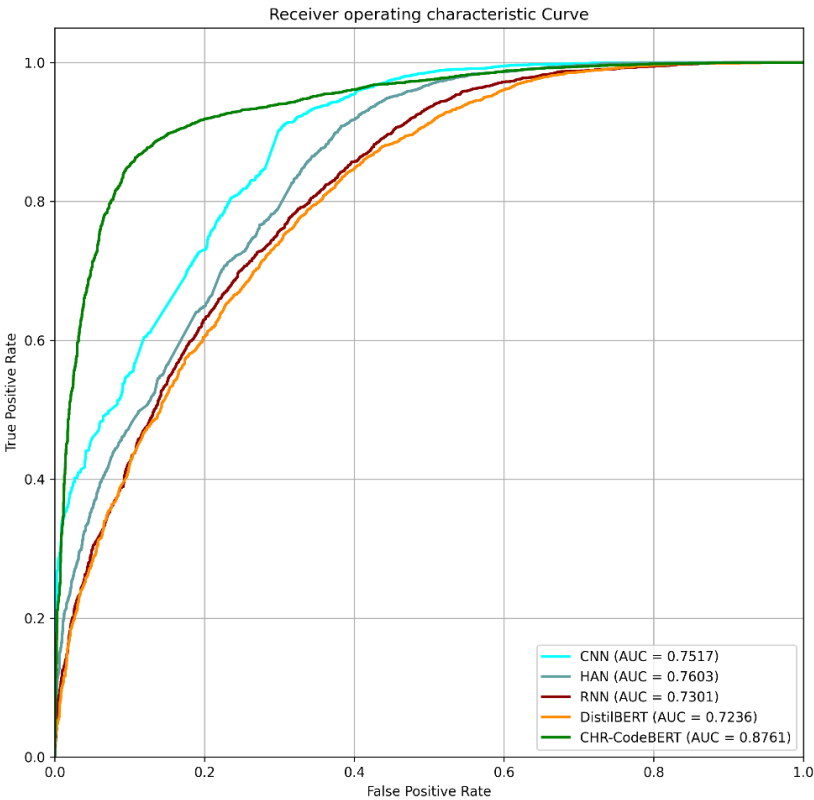


图 5.3 不同学习模型在数据集（2）的 ROC 曲线对比

表 5.7 给出了不同学习模型在数据集（2）进行漏洞检测性能评估的结果对比。在实验数据集（2）的实验效果上，CNN 和 HAN 较 RNN 和 DistilBERT 在整体上具有更有利的指标，可见在数据集平衡且数量较大的情况下，这两种模型的漏洞检测能力更强，具有更好的分类效果；CHR-CodeBERT 较其余模型，准确率高达 87.30%，精确率提升至 86.88%，误报率降至 14.30%，F1 分数增至 87.11%。同时，由图 5.3 可知，CHR-CodeBERT 模型 ROC 曲线的 AUC 值更高，整体呈更佳的性能。

2、CHR-CodeBERT 与 5.1 节模型对比

本实验选取的比较对象来自于文献[71]采用的一种基于 VUDENC 的漏洞检测方法，应用 word2vec 模型来识别语义相似的代码标记并提供矢量表示，然后使用长短期记忆（LSTM）网络对代码进行检测，在相同的实验环境下进行比较。具体实验结果如表 5.8、表 5.9 所示。

表 5.8 CHR-CodeBERT 模型和文献[71]在数据集（1）的性能评估结果对比

方法	ACC	P	TPR	FPR	F1	AUC
VUDENC	65.52%	60.44%	61.75%	30.51%	66.32%	64.49%
CHR-CodeBERT	67.77%	68.13	63.09%	26.86%	67.77%	68.12%

表 5.9 CHR-CodeBERT 模型和文献[71]在数据集（2）的性能评估结果对比

方法	ACC	P	TPR	FPR	F1	AUC
VUDENC	66.75%	68.11%	66.05	30.24%	65.49%	66.05%
CHR-CodeBERT	87.30%	86.88%	89.53%	14.30%	87.11%	87.61%

由上述两个表格可知本文提出的一种基于多特征融合的源代码漏洞检测方法 CHR-CodeBERT 具有较优的性能，可以解决 5.1 节中对代码的表征能力不足、没有完全捕获漏洞的关键特征的问题，在两个数据集上的检测结果呈现较高的准确率，体现多特征挖掘对漏洞检测的优越性。因此该漏洞检测方法可以满足第三章中漏洞检测模块的设计需求。

5.4 本章小结

本章首先对相关漏洞检测方法的问题进行了研究和分析，提出了一种基于多特征融合的源代码漏洞检测方法 CHR-CodeBERT，该方法主要分为数据预处理、特征提取与融合、模型分类三个阶段。数据预处理是对源代码数据进行清洗操作，降低无用信息被表征而增加特征矩阵的维度；特征提取与融合是对清洗后的源代码进行 CNN、HAN 和 RNN 特征提取，通过设计的加性注意力机制对这三种特征进行融合，构成一个特征矩阵，从而全面表征漏洞特征；模型分类是将上述融合特征输入到 CodeBERT 模型进行分类，挖掘出当前检测的源代码是否有漏洞。最后，整理了两个漏洞源代码数据集，并对这两个数据集进行了大量的相关对比实验，采用准确率、精确率、误报率等性能指标作为评估，实验结果表明了本章模型的优越性。

第六章 系统实现与测试

本章将提出的 BDM-MNB 和 CHR-CodeBERT 两种漏洞检测方法应用第三章设计的基于深度学习的源代码漏洞检测系统中，首先介绍了系统的软硬件环境，然后详细阐述了后端检测模块和前端结果可视化模块，最后对嵌入第四章和第五章漏洞检测方法的系统进行了功能测试。

6.1 系统环境

6.1.1 硬件环境

为了有效调用组合学习模型和多特征融合模型对源代码漏洞实现检测，基于深度学习的源代码漏洞检测系统需要一定的硬件环境，具体参数如表 6.1 所示。

表 6.1 系统硬件环境参数表

软件开发环境		服务器环境	
属性	参数	属性	参数
设备型号	Intel(R) Core(TM) i7-10750H	设备型号	阿里云服务器
操作系统	Win10 64 位	操作系统	Ubuntu Server 18.04.1 LTS 64 位
CPU	@ 2.60GHz 2.59 GHz	CPU	Intel Core i7-9750H 2.60GHz
GPU	NVIDIA GeForce GTX 1650	GPU	/
内存	16.00GB	内存	4.00GB

6.1.2 软件环境

本系统采用前后端分离的开发方式，前端使用 Vue 框架搭建项目，开发工具使用 VScode，后端框架搭建采用 MVC 模式，使用 Python 语言，选用 Django 框架，开发工具使用 Pycharm。数据库使用 MYSQL 服务器，缓存采用 Redis 服务器，系统具体信息如表 6.2 所示。

表 6.2 系统软件环境参数表

软件功能	软件名称	软件版本
前端开发	JavaScript	1.8.5
前端框架	Vue	3.2
JavaScript 运行环境	node	16.14.2
node.js 包管理工具	npm	8.5.0

后端开发	Pycharm	2019.2.4
后端框架	Django	3.2.13
后端开发语言	Python	3.7.0
数据库	MYSQL	5.8.0
弹性数据仓储	snowflake	0.0.3
缓存	Redis	2.8.9
版本控制	Git	2.34.1

6.2 系统方法实现

6.2.1 后端检测模块

后端检测模块是在获取前端上传的源代码文件后进行漏洞检测，对源代码进行预处理、特征工程、特征选择等一系列操作，输入到构建的深度学习模型进行检测，将当前检测结果和历史检测结果保存到数据库，同时传送到前端进行可视化。具体流程如图 6.1 所示。

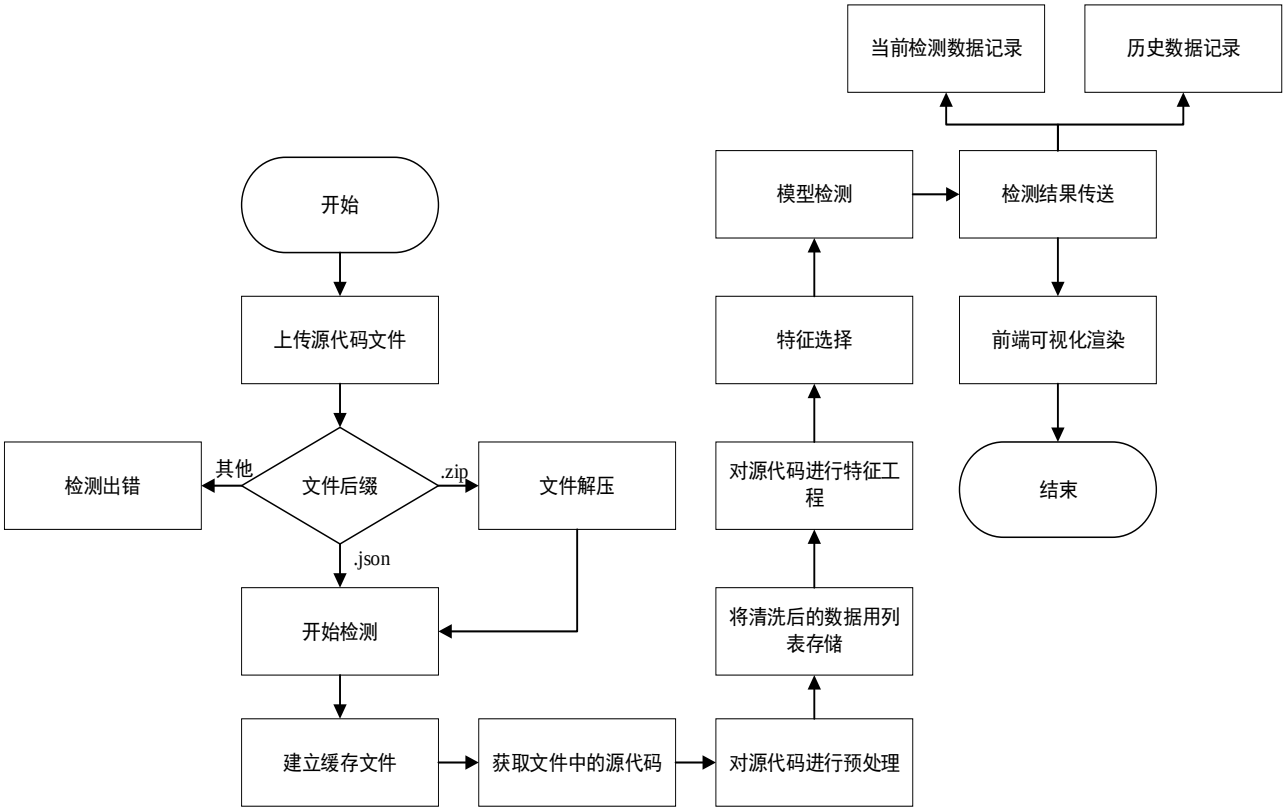


图 6.1 系统后端检测模块流程图

后端检测模块的类图如图 6.2 所示，在获取到前端成功上传的源代码文件后，若是单文件则直接检测，若是压缩文件则解压后检测，调用检测模型 Model 的方法进行检测，检测完成后将实例化 CurrentMapper 和 HistoryMapper 的对象，实现当前数据和历史数据的存储。

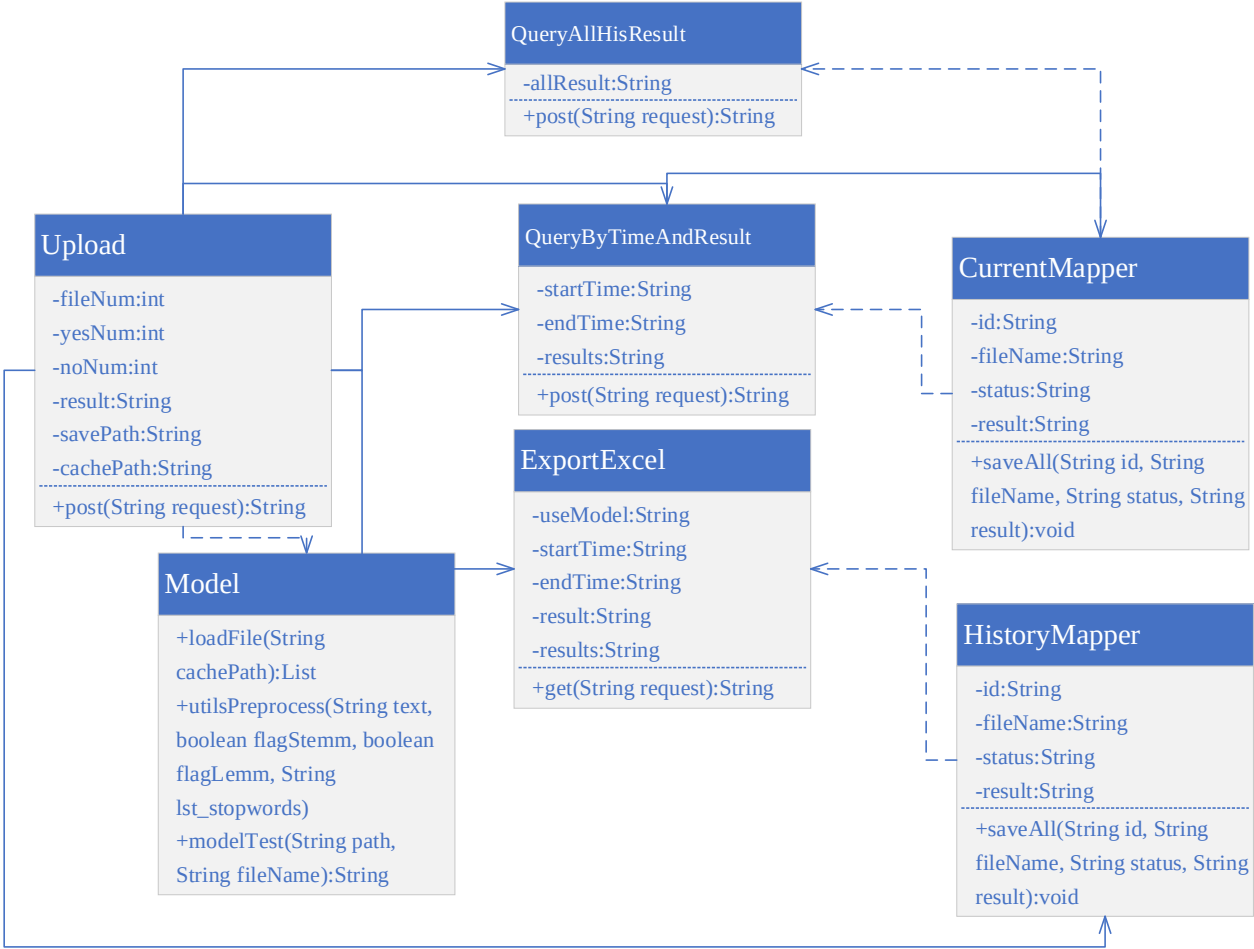


图 6.2 系统后端检测模块类图

6.2.2 前端结果可视化模块

用户上传待检测的源代码文件，若文件后缀名是.json/.zip 则上传成功，若文件后缀名为其他则上传失败，需要重新上传格式正确的源代码文件。开始检测源代码时会显示检测进度，根据后端传来的检测结果，将分为柱形图、圆饼图和检测数据的可视化，如果检测过程中有问题，可在异常监控中查看。对于系统检测过的源代码文件，使用数据库保存了历史数据，用户可点击导出按钮成 Excel 表格查看。具体流程如图 6.3 所示。

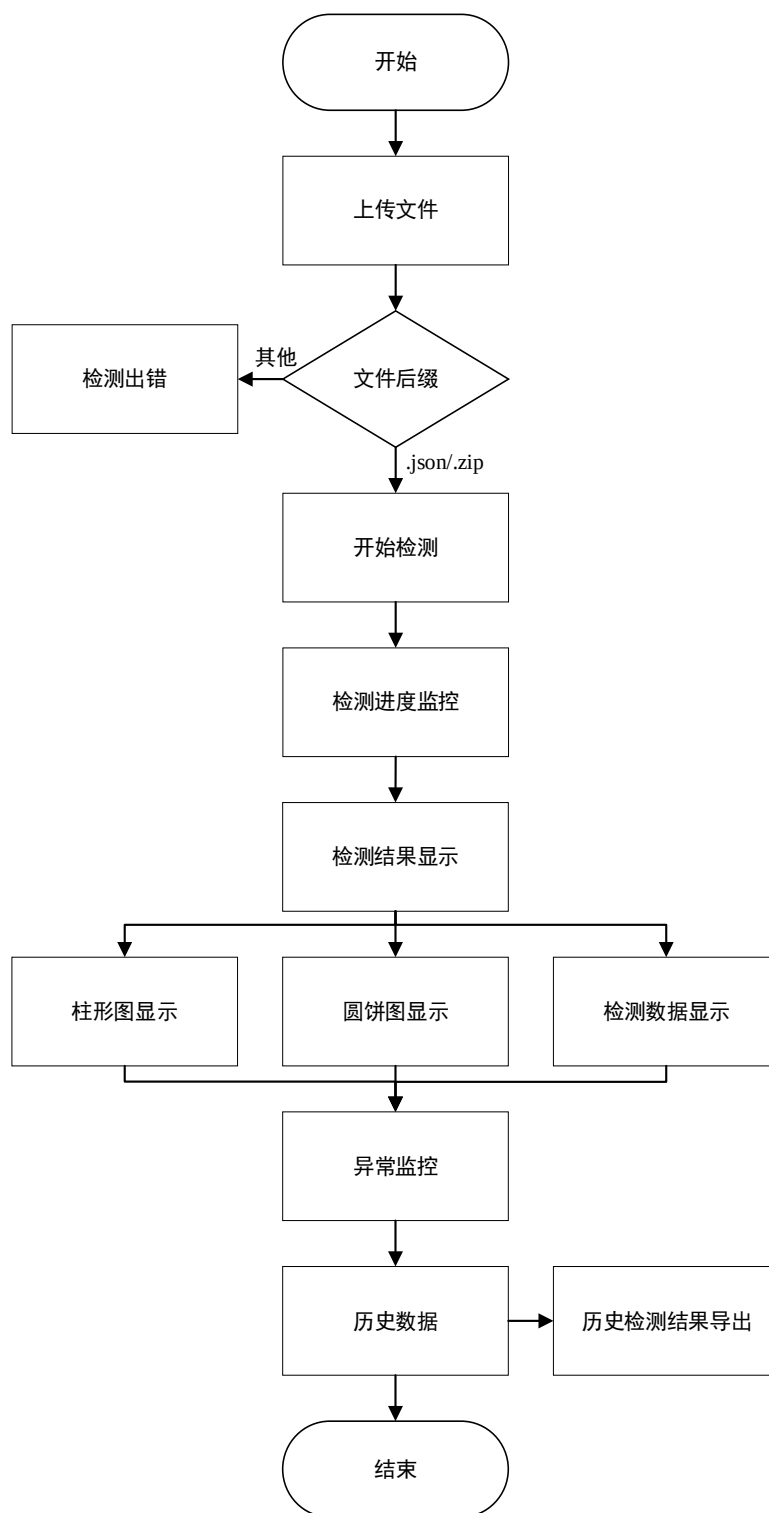


图 6.3 系统前端模块流程图

6.3 系统功能测试

6.3.1 文件上传

首先是文件上传功能,用户可以上传单文件或压缩文件,单文件源代码的后缀名为.json,

压缩文件源代码的后缀名是.zip，它是多个单文件源代码的压缩包，需要注意的是压缩包内只能有文件，不能有文件夹，否则会解压出错。

(1) 单文件源代码检测

点击文件上传按钮上传单个或多个文件缀名为.json 的源代码文件，示意图如图 6.4 所示，检测成功后会出现如图 6.5 所示的结果。

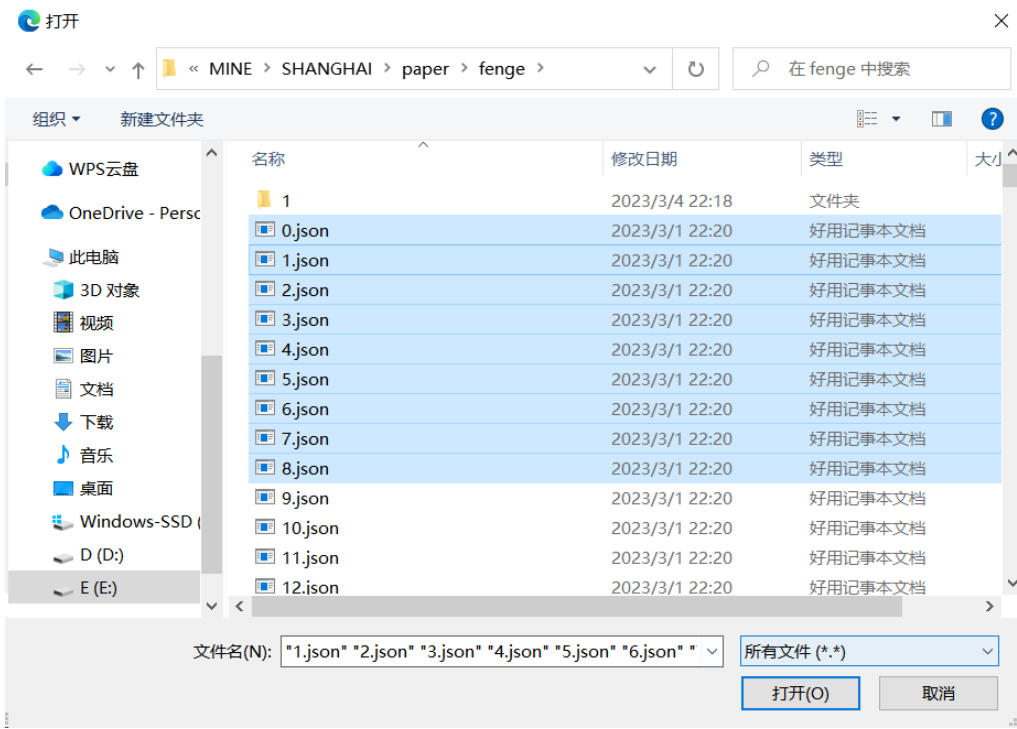


图 6.4 单/多个.json 文件上传示意图



图 6.5 单/多个.json 文件检测结果图

(2) 压缩文件源代码检测

点击上传文件后上传后缀名为.zip 的压缩文件，压缩文件内是后缀名为.json 的源代码文件，不能有文件夹，示意图如图 6.6 所示。检测结果如图 6.7 所示。

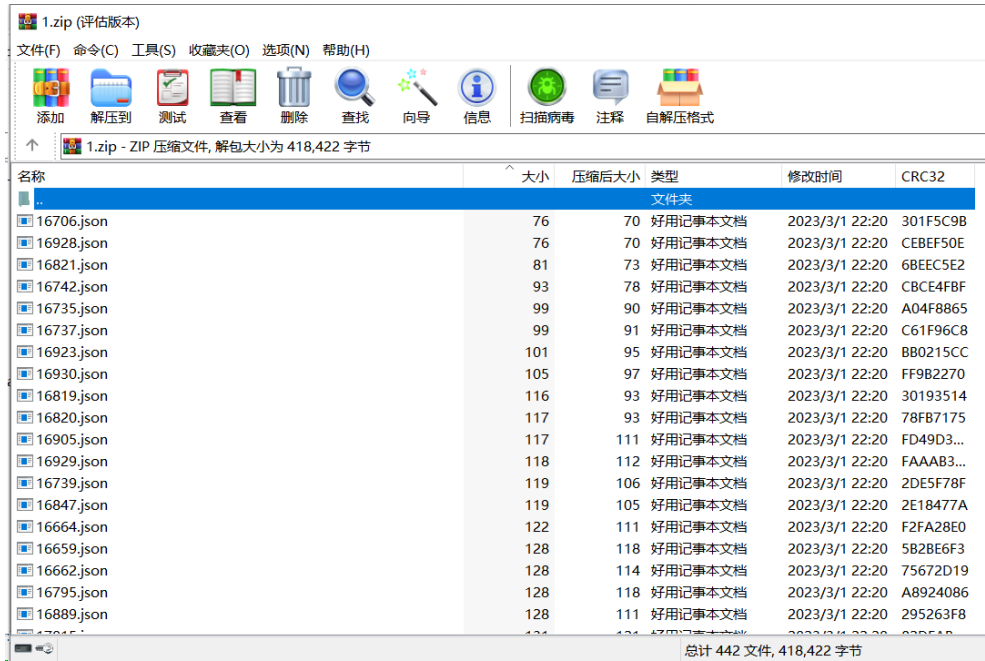


图 6.6 后缀名为.zip 的压缩文件内容示意图



图 6.7 后缀名为.zip 的多源代码文件检测结果

6.3.2 源代码数据预处理

在获取到源代码文件后，需要对其进行数据预处理操作，将所有英文字母转换成小写字母，删除多余的空行、注释行和标点符号，根据下载好的停用词词包删除不必要的停用词，将清洗好的源代码使用列表 List 进行存储。

6.3.3 源代码漏洞检测

将存储后的源代码构建特征工程，提取 TF-IDF 特征、文本特征等，进行特征选择和降维，最后形成可以直接输入到检测模型的特征向量。将特征向量作为系统后端中 DistilBert+LSTM 和多项朴素贝叶斯模型、多特征融合模型的输入进行检测，检测完毕后将检测结果发送到前

端可视化。

6.3.4 漏洞检测结果存储及可视化

(1) 柱形统计图

柱形统计图展示已检测源代码文件的数量、无漏洞文件数量和有漏洞文件数量，结果如图 6.8 所示。

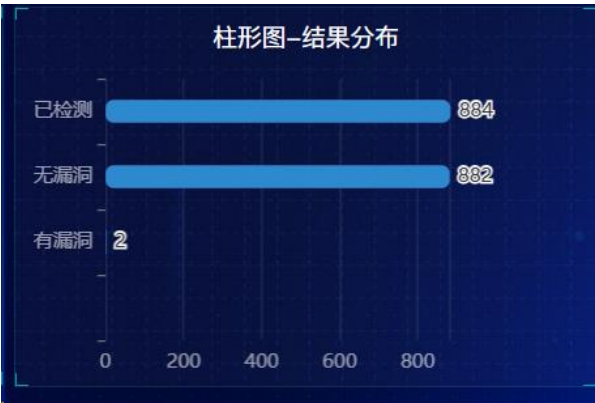


图 6.8 柱形统计图

(2) 圆饼图

圆饼图展示有漏洞文件数量和无漏洞文件数量，点击饼图可显示相应结果的百分比和数值，结果如图 6.9 所示。



图 6.8 柱形统计图

(3) 检测结果数据展示

检测结果数据展示上传源代码文件中已检测的文件数量、未检测文件数量、有漏洞文件数量和无漏洞文件数量。同时，数字显示下方展示更详细的数值供用户查看，结果如图 6.9 所示。



图 6.9 检测结果数据展示

(4) 异常监控显示

异常监控区域展示检测失败的源代码文件数量、检测文件的耗时、开始检测时间和结束检测时间，结果如图 6.10 所示。



图 6.10 异常监控显示图

(5) 历史记录展示和导出

历史记录展示之前检测的源代码文件的结果，用户可输入特定的时间范围查询历史检测结果，显示字段为文件编号、文件名、检测时间和检测结果，结果为“1”则表示有漏洞，结果为“0”则表示无漏洞，如图 6.11 所示。用户点击导出按钮可导出历史检测结果，由一个 Excel 表格呈现，结果如图 6.12 所示。



图 6.11 历史记录展示图

	A	B	C	D	
1	编号	文件名	检测时间	结果	
2	4729387932987691009	0. json	2023-03-03 22:59:12	1	
3	4729388188051705857	1. json	2023-03-03 23:00:11	1	
4	4729388188173340673	2. json	2023-03-03 23:00:11	0	
5	4729388188269809665	4. json	2023-03-03 23:00:11	0	
6	4729388188269809666	3. json	2023-03-03 23:00:11	0	
7	4729388188286586881	5. json	2023-03-03 23:00:11	0	
8	4729388188328529921	6. json	2023-03-03 23:00:11	0	
9	4729388188567605249	7. json	2023-03-03 23:00:12	0	
10	4729388188584382465	8. json	2023-03-03 23:00:12	0	
11	4729388188634714113	9. json	2023-03-03 23:00:12	1	
12	4729388188685045761	10. json	2023-03-03 23:00:12	1	
13					

图 6.12 历史结果导出展示图

6.4 本章小结

本章介绍了基于深度学习的源代码漏洞检测系统的具体实现。首先对系统的硬件环境和软件进行了介绍，系统采用了前后端框架与请求/响应设计，前端使用 Vue 框架搭建，并用 npm 进行相关包的管理，后端使用 Django 框架搭建，并用 Pycharm 编写代码和管理相关包。在系统总体设计的基础上，细化每个模块的功能，并对基于 DBM-MNB 和 CHR-CodeBERT 漏洞检测方法的系统进行了功能测试，测试结果达到预期的效果。

第七章 总结与展望

7.1 总结

本文设计并实现了基于深度学习的源代码漏洞检测系统，系统采用两个漏洞检测模型，可在后端模型调用的地方更改模型加载的路径。本文共设计了两种漏洞检测方法，一种是基于 DistilBert-LSTM 与多项朴素贝叶斯的源代码漏洞检测方法 DBM-MNB，另一种是基于多特征融合的源代码漏洞检测方法 CHR-CodeBERT，两个检测方法均可使用，可以根据用户对漏洞表征的需求实现切换，最终实现漏洞检测的需求。本文所作的主要工作有：

(1) 概述了基于深度学习的源代码漏洞检测系统的相关理论技术以及国内外的研究学者针对基于深度学习的漏洞检测方法的研究现状。使用 UML 建模思想，对系统的总体架构进行了设计，同时阐述了系统的主要功能和子模块设计。

(2) 聚焦源代码漏洞检测的深度，提出了一种基于 DBM-MNB 的源代码漏洞检测方法。首先进行了问题分析，阐述了模型的总体框架，详细介绍了漏洞检测模型，通过模型挖掘漏洞的局部关键特征和全局时间特征，得出漏洞的存在性概率；针对漏洞检测过程中的难样本，通过多项朴素贝叶斯进行优化检测，该模型使用 TF-IDF 矢量化器进行数据预处理，并执行卡方检验进行特征选择，将所得结果输出至多项朴素贝叶斯分类器中进行检测，以获得最终的漏洞检测结果。最后进行了对比实验，验证了 DBM-MNB 方法的有效性。

(3) 聚焦源代码漏洞表征的完整度，提出了基于 CHR-CodeBERT 的源代码漏洞检测方法。首先进行了问题分析，阐述了模型的总体框架，详细介绍了特征提取和融合的过程，模型提取 CNN、HAN 和 RNN 特征，根据设计的加性注意力机制模块进行特征融合成一个特征矩阵。同时，对预处理后的数据进行添加特殊标记、生成掩码等操作构建一个特征矩阵。将以上两步获得的两个特征矩阵作为 CodeBERT 的模型输入，添加平均池化层和两个全连接层实现对源代码中漏洞的检测。最后进行了对比实验，验证了 CHR-CodeBERT 在多个数据集上的有效性。

(4) 设计并实现了基于深度学习的源代码漏洞检测系统，并对主要功能进行了测试。

7.2 展望

本文设计的基于深度学习的源代码漏洞检测系统随着深度学习的日益发展，该系统在实

实际应用过程中存在许多不足之处:

(1) 本文设计的系统功能较为简单, 在用户友好互动上仍需改进, 同时系统的压力测试进行的不完整, 当系统使用数量和源代码检测数量达到一定规模时, 检测模型的速度会使系统产生算力不足的风险, 后续可以对系统的部署方式进行更改, 例如采用分布式部署方式。

(2) 本文提出的 DBM-MNB 模型重点捕捉漏洞特征的时间特征和依赖关系, 对于漏洞的空间特征没有针对性捕获, 后续研究可以在特征提取上关注漏洞特征的空间关系, 将语义和空间位置的相关性增加到模型中, 提升模型漏洞检测的准确性能。

(3) 本文提出的 CHR-CodeBERT 模型均是基于源代码文本信息进行表征, 对于漏洞函数之间的顺序关系、跳转关系等捕获的不明显, 后续研究可以将这些关系使用其他表征方式呈现, 结合 CHR 特征进行漏洞检测, 增加模型的表征能力。

(4) 本文提出的两种漏洞检测方法主要是检测源代码是否存在漏洞, 本质上是二分类问题, 后续研究可以整理更细致的漏洞数据集, 实现漏洞的多分类, 即检测源代码中的漏洞所属类别, 增强漏洞检测的能力。

参考文献

- [1] 郑尧文, 文辉, 程凯, 等. 物联网设备漏洞检测技术研究综述[J]. 信息安全学报, 2019, 4(05): 61-75.
- [2] 刘嘉勇, 韩家璇, 黄诚. 源代码漏洞静态分析技术[J]. 信息安全学报, 2022, 7(04): 100-113.
- [3] Konstantinos K, Theodores T. SQL-IDS: a specification-based approach for SQL-injection detection[C]. In Proceedings of the 2008 ACM symposium on Applied computing Association for Computing Machinery, New York, NY, USA, 2008, 2153–2158.
- [4] Jose F, Marco V, Henrique M. Testing and comparing web vulnerability scanning tools for SQL injection and XSS attacks[C]. 13th Pacific Rim International Symposium on Dependable Computing (PRDC 2007), Melbourne, VIC, Australia, 2007, pp. 365-372.
- [5] Lhee K S, Chapin S J. Buffer overflow and format string overflow vulnerabilities[J]. Software: practice and experience, 33(5), 423-460.
- [6] Huluka D, Popov O . Root cause analysis of session management and broken authentication vulnerabilities[C]. In World Congress on Internet Security (WorldCIS-2012) , 2012, pp. 82-86.
- [7] Li Z, Zou D, Xu S, et al. Vuldelocator: a deep learning-based fine-grained vulnerability detector. IEEE Transactions on Dependable and Secure Computing, 19(4), 2821-2837.
- [8] Flawfinder[EB/OL]. <https://www.dwheeler.com/flawfinder>, 2019.
- [9] RATS[EB/OL]. <https://security.web.cern.ch/security/recommendations/en/codetools/rats.shtml>, 2019.
- [10] CPPCHECK[EB/OL]. <http://cppcheck.sourceforge.net>, 2019.
- [11] SPOTBUGS[EB/OL]. <https://spotbugs.readthedocs.io/en/stable/>, 2019.
- [12] PMD[EB/OL]. <https://pmd.github.io/latest/index.html>, 2019.
- [13] 李韵, 黄辰林, 王中锋, 等. 基于机器学习的软件漏洞检测方法综述[J]. 软件学报, 2020, 31(07): 2040-2061.
- [14] Wu F, Wang J, Liu J, et al. Vulnerability detection with deep learning[C]. in Proceedings of the 2017 3rd IEEE International Conference on Computer and Communications (ICCC), IEEE, 1298–1302.
- [15] Grieco G, Grinblat G L, Uzal L, et al. Toward large-scale vulnerability discovery using machine learning[C]. in Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy, 2016, 85–96.
- [16] Wang Y, Wu Z, Wei Q, et al. NeuFuzz: efficient fuzzing with deep neural network[J]. IEEE Access, vol. 7, pp. 36340–36352, 2019.
- [17] Zaddach J, Bruno L, Francillon A, et al. AVATAR: A framework to support dynamic security analysis of embedded systems' firmwares[C]. In Proceedings of the Network and Distributed System Security (NDSS) Symposium, San Diego, CA, USA, 2014, 23–26.
- [18] Kammerstetter M, Platzer C, Kastner W. Prospect: Peripheral proxying supported embedded code testing[C]. In Proceedings of the 9th ACM Symposium on Information, Computer and Communications Security, Kyoto, Japan, 2014, 3–6.
- [19] Koscher K, Kohno T, Molnar D. SURROGATES: Enabling near-real-time dynamic analyses of embedded systems. In Proceedings of the 9th USENIX Workshop on Offensive Technologies (WOOT 15), Washington, DC, USA, 2015, 10–11.
- [20] Muench M, Nisi D, Francillon A, et al. Avatar 2: A multi-target orchestration platform. In Proceedings of the Workshop on Binary Analysis Research (colocated with NDSS Symposium), San Diego, CA, USA, 2018.
- [21] Chen D D, Woo M, Brumley D, et al. Towards automated dynamic analysis for linux-based embedded firmware[C]. In Proceedings of the Network and Distributed System Security (NDSS) Symposium, San Diego, CA, USA, 2016, 21–24.
- [22] Bellard F. QEMU, a fast and portable dynamic translator. In Proceedings of the USENIX Annual Technical Conference, Anaheim, CA, USA, 2005, 10–15.
- [23] 邓泉, 叶蔚, 谢睿, 等. 基于深度学习的源代码缺陷检测研究综述[J]. 软件学报, 2023, 34(02): 625-654.

- [24] 肖添明, 管剑波, 蹇松雷, 等. 基于代码属性图和 Bi-GRU 的软件脆弱性检测方法[J]. 计算机研究与发展, 2021, 58(08): 1668-1685.
- [25] 文敏, 王荣存, 姜淑娟. 基于关系图卷积网络的源代码漏洞检测[J]. 计算机应用, 2022, 42(06): 1814-1821.
- [26] 梁树彬, 郑力, 钟杰, 胡勇. 基于卷积神经网络的源代码漏洞检测模型[J]. 通信技术, 2022, 55(04): 493-499.
- [27] Li L, Feng H, Zhuang W, et al. CCleaner: a deep learning-based clone detection approach[C]. in Proceedings of the 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME), 249–260.
- [28] ANTLR[EB/OL], <http://www.antlr.org/>.
- [29] Use JDT ASTParser to Parse Single[EB/OL], <https://www.programcreek.com/2011/11/use-jdt-astparser-to-parsejava-file/>.
- [30] Marastoni N, Giacobazzi R, Dalla P M. A deep learning approach to program similarity[C]. in Proceedings of the 1st International Workshop on Machine Learning and Software Engineering in Symbiosis, ACM, Montpellier, France, 26–35.
- [31] ChristianCollberg, Tigress C. <http://tigress.cs.arizona.edu/>, 2020
- [32] Tufano M, Watson C, Bavota G, et al. Deep learning similarities from different representations of source code[C]. in Proceedings of the 2018 IEEE/ACM 15th International Conference on Mining Software Repositories (MSR), IEEE, Gothenburg, Sweden, 542–553.
- [33] Zhang R, Li W, Tong M. Review of deep learning[J]. Information and Control, 47(4), 385–397, 2018.
- [34] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]. Advances in neural information processing systems, 2017, 30.
- [35] Jacob D, Chang W M, Kenton L, et al. BERT: Pre-training of deep bidirectional transformers for language understanding[C]. In Proc. the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 4171–4186.
- [36] Alec R, Karthik N, Tim S, et al. Improvin language understanding by generative pre-Training[C]. In Proc. of Findings of the Association for Computational Linguistics: EMNLP2020, 1536–1547.
- [37] Feng Z, Guo D, Tang D, et al. Codebert: A pre-trained model for programming and natural languages[J]. arXiv preprint arXiv:2002.08155.
- [38] Guo D Y, Ren S, Lu S, et al. GraphCodeBERT: Pre-training code representations with data flow. CoRR abs/2009. 08366 (2020). arXiv: 2009. 08366.
- [39] Mike L, Liu Y H, Naman G, et al. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. CoRR abs/1910.13461 (2019). arXiv: 1910. 13461.
- [40] Wasi U A, Saikat C, Baishakhi R, et al. Unified pre-training for program understanding and generation. CoRR abs/2103.06333 (2021). arXiv: 2103. 06333.
- [41] Noah Z, Wu S. Security vulnerability detection using deep learning natural language processing. arXiv: cs.CR/2105. 02388.
- [42] Mohammad S, Mostofa P, Raul P, et al. Megatron-LM: Training multi-billion parameter language models using model parallelism. arXiv: cs.CL/1909. 08053.
- [43] 顾绵雪, 孙鸿宇, 韩丹, 等. 基于深度学习的软件安全漏洞检测[J]. 计算机研究与发展, 2021, 58(10): 2140-2162.
- [44] Bosu A, Carver J C, Hafiz M, et al. Identifying the characteristics of vulnerable codechanges: An empirical study[C]. in Proc. 22nd ACM SIGSOFT Int. Symp. Found. Softw.Eng. FSE, 2014, 257–268.
- [45] Chernis B, Verma R. Machine learning methods for software vulnerability detection[C]. in Proc. 4th ACM Int. Workshop Secur. Privacy Analytics IWSPA, 2018, 31–39.
- [46] Chen L, Ye W, Zhang W. Capturing source code semantics via treebased convolutionover API-enhanced AST[C]. in Proc. 16th ACM Int. Conf.Comput. Frontiers, Apr. 2019, 174–182.

- [47] Shin Y, Meneely A, Williams L. Evaluating complexity, codechurn, and developer activity metrics as indicators of software vulnerabilities[J]. IEEE TransSoftw, 2011, 37(6), 772–787.
- [48] Zhang J, Wang X, Zhang H. A novel neural source code representation based on abstract syntax tree[C]. in Proc. IEEE/ACM 41st Int. Conf. Softw. Eng. (ICSE), May 2019, 783–794.
- [49] Zhou Y, Liu S, Siow J. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks[C]. in Proc. Adv. Neural Inf. Process. Syst., New York, NY, USA: Curran Associates, 2019, 10197–10207.
- [50] Alon U, Zilberstein M, Levy O, et al. Code2vec: Learning distributed representations of code[J]. ACM Program Lang, 2019, 3, 1–29.
- [51] Dahse J, Schwenk J. RIPS-A static source code analyser for vulnerabilities in PHP scripts[J]. In Seminar Work (Seminar Çalışması); Horst Görtz Institute Ruhr-University Bochum: Bochum, Germany, 2010.
- [52] Source Code Security Audit[EB/OL]. <https://github.com/WhaleShark-Team/cobra>, 2020.
- [53] 陈皓, 易平. 基于图神经网络的代码漏洞检测方法[J]. 网络与信息安全学报, 2021, 7(03): 37-45.
- [54] 邹德清, 李响, 黄敏桓, 宋翔, 李浩, 李伟明. 基于图结构源代码切片的智能化漏洞检测系统[J]. 网络与信息安全学报, 2021, 7(05): 113-122.
- [55] 段旭, 吴敬征, 罗天悦, 等. 基于代码属性图及注意力双向 LSTM 的漏洞检测方法[J]. 软件学报, 2020, 31(11): 3404-3420.
- [56] Guo N, Li X, Yin H, et al. VulHunter: An automated vulnerability detection system based on deep learning and bytecode[C]. In Proceedings of the International Conference on Information and Communications Security, 2019, 199–218.
- [57] Fang Y, Han S, Huang C, et al. TAP: A static analysis model for PHP vulnerabilities based on token and deep learning technology. PLoS ONE 2019, 14, e0225196.
- [58] Liu S, Lin G, Qu L, et al. CD-VulD: Cross-domain vulnerability discovery based on deep domain adaptation[J]. IEEE Trans Dependable Secur. Comput, 2020, 1.
- [59] Walden J, Stuckman J, Scandariato R. Predicting vulnerable components: Software metrics vs text mining[C]. in Proc. IEEE 25th Int. Symp. Softw. Rel. Eng., 2014, 23–33.
- [60] Hindle A, Barr E T, Su Z, et al. On the naturalness of software[C]. in Proc. 34th Int. Conf. Softw. Eng. (ICSE), Jun. 2012, 837–847.
- [61] Efstathiou V, Spinellis D. Semantic source code models using identifier embeddings[C]. in Proc. IEEE/ACM 16th Int. Conf. Mining Softw. Repositories (MSR), May 2019, 29–33.
- [62] Russell R, Kim L, Hamilton L. Automated vulnerability detection in source code using deep representation learning[C]. in Proc. 17th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA), Dec. 2018, 757–762.
- [63] 王剑, 匡洪宇, 李瑞林, 等. 基于 CNN-GAP 可解释性模型的软件源码漏洞检测方法[J]. 电子与信息学报, 2022, 44(07): 2568-2575.
- [64] 杨宏宇, 应乐意, 张良. 基于结构化文本及代码度量的漏洞检测方法[J]. 湖南大学学报(自然科学版), 2022, 49(04): 58-68.
- [65] Huang G, Li Y, Wang Q, et al. Automatic classification method for software vulnerability based on deep neural network[J]. in IEEE Access, 2019, 7, 28291-28298.
- [66] Ranhan G, Talukder K, Mithila S. An empirical study to detect cyberbullying with TF-IDF and machine learning algorithms[C]. 2021 International Conference on Electronics, Communications and Information Technology (ICECIT), 2021, 1-4.
- [67] Pu K, Ma L. Incremental Computation of Information Gain in Temporal Relational Streams[C]. 2022 IEEE 23rd International Conference on Information Reuse and Integration for Data Science (IRI), 2022, 234-235.
- [68] Singh B, Jain B, Das B. A framework for asymmetrical DNN modularization for optimal loading[C]. 2021 International Joint Conference on Neural Networks (IJCNN), 2021, 1-8.
- [69] Wang L, Li X, Wang R, et al. PreNNsem: A heterogeneous ensemble learning framework for vulnerability

- detection in software[J]. Applied Sciences, 2020, 10(22): 7954-7969.
- [70] Peng H, Mou L L, Li G, et al. Building program vector representations for deep learning [C]. Proc of the 8th Int Conf on Knowledge Science, Engineering and Management, 2015, 547-553.
- [71] Wartschinski L, Noller Y, Vogel T. et al. Vudenc: Vulnerability detection with deep learning on a natural codebase for python[J]. Information and Software Technology, 2022, 144: 106809.
- [72] Marashdih A W, Zaaba Z F, Suwais K. Predicting input validation vulnerabilities based on minimal SSA features and machine learning[J]. Journal of King Saud University-Computer and Information Sciences, 2022, 34(10): 9311-9331.
- [73] Jia H F, Yi L, Shao H W et al. A C/C++ code vulnerability dataset with code changes and CVE summaries[C] In MSR '20: The 17th International Conference on Mining Software Repositories, 2020.
- [74] Thapa C, Jang S I, Ahmed M E, et al. Transformer-based language models for software vulnerability detection: performance, model's security and platforms.arXiv preprint arXiv, 2022, 2204.03214.